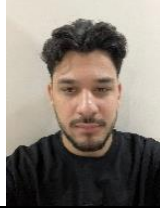
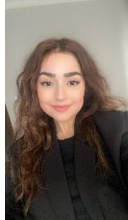




CVISION

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Student Number	Name and Surname	Role	Photo
230303033	Fatih Çatalçam	Leader	
220303036	Hüseyin Kaya	Developer	
230303034	Emirhan Selcen	Developer	
220303023	Ada Şevval Sari	Tester	

Document Name	CVision_READ3.docx
Version No	1.3
Version Date	13.05.2026

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

DOCUMENT APPROVAL INFORMATION

Contributors	Fatih Çatalçam (Leader)	Sign ☒
	Ada Şevval Sari (Tester)	☒
	Hüseyin Kaya (If needed)	☒
	Emirhan Selcen (If needed)	☒
		☐
Reviewer-1 (LLM)	Claude	☒
Reviewer-2	Ada Şevval Sari	☒
Reviewer-3	Emirhan Selcen	☒
Stakeholder (Instructor)	Haluk Gümüşkaya	☐

CHANGE RECORDS

Version	Comments	Date
1.0	Initial release full requirements elicitation and analysis document	20.04.2026
1.1	Updated diagrams (System Architecture, Deployment, Sequence) redrawn in Visual Paradigm	27.04.2026
1.2	Small readjustments and updated pictures	5.05.2026
1.3	Added secure Password Reset and Gmail SMTP email integration forgot-password request, reset-code verification, password reset execution, EmailService, reset-code fields in User model, password history, updated scenarios, use cases, ER model, UI screens, acceptance criteria, traceability, and conclusion.	13.05.2026

DISTRIBUTION RECORDS

Distribution No	Version No	Comments	Distribution Method
1	1.0	First submission	Upload to Google Classroom
2	1.1	Updated diagrams submission	Upload to Google Classroom
3	1.2	Updated Pictures	Upload to Google Classroom
4	1.3	Forgot password feature and Gmail SMTP integration added	Upload to Google Classroom

Contents

1. INTRODUCTION	5
1.1. Objectives of Requirements Engineering.....	5
1.2. Purpose of the Document.....	5
1.3. Project Scope	5
1.4. Objectives and Success Criteria	6
1.5. Stakeholders	7
1.6. System Context	7
2. REQUIREMENTS ELICITATION	8
2.1. Elicitation Methods and Tools	8
2.2. High-Level Requirements Summary	8
2.3. Functional Requirements	10
2.4. Non-Functional Requirements.....	13
2.4.1. Quality Requirements: Specific Quality Requirements	13
2.4.2. Quality Requirements: Performance Requirements	15
2.4.3. Constraints	16
2.5. Assumptions.....	18
3. REQUIREMENTS ANALYSIS	19
3.1. Scenarios.....	19
3.2. Use Case Models.....	21
3.3. Data Models (Entity-Relationship (ER Diagrams)).....	24
3.4. Problem Domain Class/Object Models.....	27
3.5. Dynamic Models	29
3.6. Component, Package and Deployment Models.....	31
3.7. System Architecture	33
3.8. User Interfaces	33
33	
3.9. Minimum Requirements for This Project.....	38
4. REQUIREMENTS VALIDATION.....	38
4.1. Validation Approach	38
4.2. Acceptance Criteria	40
5. TRACEABILITY.....	41
6. CONCLUSION	45
7. REFERENCES.....	46
8. DESCRIPTIONS, ABBREVIATIONS, AND DICTIONARY	47
8.1. Descriptions	47
8.2. English and Turkish Abbreviations	48
8.3. Turkish Dictionary	48

1. INTRODUCTION

1.1. Objectives of Requirements Engineering

Requirements Engineering (RE) is a systematic process that encompasses the identification, documentation, validation, and management of the needs and constraints of all stakeholders who are to interact with a software system. For the CVision project, RE is critical because the system bridges multiple technical domains: secure user authentication, file parsing, AI-driven text analysis, and payment processing.

The primary RE activities undertaken in this document are: (1) Elicitation - gathering requirements from stakeholder analysis and scenario modeling; (2) Analysis - structuring and prioritizing those requirements; (3) Specification - expressing requirements in a clear, unambiguous form; (4) Validation - verifying that requirements accurately reflect stakeholder needs; and (5) Management - maintaining traceability across all artifact types. Additionally, a secure three-step password reset workflow was implemented, allowing users to recover account access via a time-limited email verification code delivered through Gmail SMTP, with enumeration-safe API responses ensuring no user-account information is leaked during the process.

1.2. Purpose of the Document

This document is the authoritative requirements baseline for the CVision platform. It targets the following audiences:

- Development Team: Provides a precise functional and non-functional specification to guide design and implementation decisions.
- Quality Assurance Engineers: Serves as the basis for test case derivation and acceptance criteria.
- Project Stakeholders & Instructor: Demonstrates the completeness, consistency, and feasibility of the envisioned system.
- Future Maintainers: Acts as a living reference for understanding system behavior and constraints.

1.3. Project Scope

In-Scope:

- User registration, authentication, and JWT-based session management
- CV file upload and secure storage (PDF and TXT formats)
- Automated multi-step analysis pipeline: section detection, skill extraction, ATS compliance checking, keyword scoring, experience evaluation, score calculation, and suggestion generation
- AI-generated summary via OpenAI GPT API (Premium users only)
- Career role recommendation engine using keyword and skill matching
- Subscription management via Stripe / lyzico payment gateway webhooks
- Admin dashboard for user management, log inspection, and system monitoring
- Role-based access control (Guest, Registered User, Premium User, Admin)
- Password reset via email using 5-character case-sensitive alphanumeric verification codes, 10-minute expiry, 5-attempt limit, bcrypt password update, password history control, and Gmail SMTP integration
- RESTful API backend (FastAPI) with React/Vite frontend

Out-of-Scope:

- Native mobile applications (iOS/Android)
- Real-time live collaboration or shared CV editing
- Resume generation or automated CV rewriting¹
- Third-party HR system integrations (ATS vendor APIs)
- Offline or edge-device processing modes
- SMS-based password recovery or third-party identity provider recovery flows

1.4. Objectives and Success Criteria

The table below defines the measurable objectives of the CVision platform:

Objective	Measurable Success Criteria
Automated CV Analysis	Analysis pipeline completes in ≤ 10 seconds for a standard 2-page PDF on production hardware
ATS Compatibility Scoring	ATS score accurately reflects ≥ 8 deterministic checks with documented pass/fail criteria
Career Recommendation Quality	Top-3 role recommendations have a match score ≥ 0.5 for a well-formed CV
Platform Availability	System uptime $\geq 99.5\%$ in production environment (excluding scheduled maintenance)
Secure Authentication	All protected endpoints return HTTP 401 without a valid JWT; token expires in 30 minutes
Payment Integration	Stripe webhook signature verification passes 100% of valid events; fails gracefully on tampered ones
Scalable Data Model	Alembic migrations run without data loss across all defined schema versions
Secure Password Recovery	A registered user can request a reset code, receive a 5-character case-sensitive alphanumeric code by email, verify the code within 10 minutes, and successfully log in with the new password after reset. Invalid or expired codes are rejected, and verification is limited to 5 attempts.

1.5. Stakeholders

Stakeholder	Role	Importance (Critical/Major/Minor)	Expectations
End User (Job Seeker)	Primary system user	Critical	Fast, accurate CV analysis; actionable suggestions; career guidance
Premium Subscriber	Paying end user	Critical	AI summary, unlimited analyses, priority support
Platform Administrator	System operator	Major	User management, audit logs, subscription oversight
Development Team	Builders & maintainers	Critical	Clear, testable requirements; stable architecture
Instructor / Evaluator	Academic stakeholder	Major	Standards-compliant documentation; completeness
OpenAI API	External service provider	Major	Well-formed prompts; quota management
Stripe / Iyzico	Payment gateway	Major	Secure webhook handling; correct event processing
Gmail SMTP / Email Service	External email delivery service.	Major.	Reliable delivery of password reset verification codes via SMTP/TLS.

1.6. System Context

CVision operates as a three-tier SaaS platform. The boundary of the system includes the React/Vite frontend, the FastAPI backend, the SQLite/PostgreSQL database, and the local file storage directory. External actors interacting across the system boundary are: (1) authenticated human users via HTTPS, (2) the OpenAI GPT API for AI-powered text summarization, (3) the Stripe/Iyzico payment gateway for subscription lifecycle management, and (4) Gmail SMTP, which is used by the EmailService to deliver password reset verification codes through SMTP/TLS on port 587.

The system does not directly control or integrate with ATS vendor platforms, employer databases, or recruitment portals. All data processed by CVision remains internal unless explicitly transmitted to OpenAI for AI summaries, the payment gateway for subscription processing, or Gmail SMTP for password reset email delivery.

2. REQUIREMENTS ELICITATION

This section presents the complete set of functional and non-functional requirements for CVision, derived from stakeholder analysis, use-case modeling, system architecture review, and iterative scenario development.

2.1. Elicitation Methods and Tools

- Stakeholder Analysis: Direct identification of user roles (Guest, Registered User, Premium User, Admin) and mapping of their interactions using the Use Case diagram.
- Use Case Modeling: The UC_CVision PlantUML diagram was used to enumerate all system interactions and derive functional requirements systematically.
- Source Code Review: The actual Python/FastAPI backend codebase (CVision source) was analyzed to ensure requirements accurately describe implemented and intended behavior.
- Password Recovery Flow Review: The implemented AuthRouter, EmailService, User model, and frontend password reset pages were reviewed to identify the forgot-password request, reset-code verification, Gmail SMTP email delivery, password reset execution, and related security requirements.
- Scenario-Based Elicitation: Narrative scenarios (normal flow, error flow) were developed for the CV upload and analysis pipeline to surface edge-case requirements.
- Architecture Review: The ARCH_CVision and DEP_CVision diagrams were analyzed to identify technical constraints and non-functional requirements.

2.2. High-Level Requirements Summary

ID	Description	Type
FR-UC-01	User registration	Functional
FR-UC-02	User login and JWT issuance	Functional
FR-UC-07	CV file upload with validation and secure storage	Functional
FR-DOC-14	Automated 7-step CV analysis pipeline	Functional
FR-UC-20	Career role recommendations based on keyword and skill matching	Functional
FR-UC-22	AI-generated CV summary via OpenAI GPT for premium users	Functional
FR-ST-23	Subscription checkout session creation	Functional
FR-ST-24	Payment webhook event processing	Functional
FR-ST-26	Admin user management	Functional
FR-UC-28	Forgot password request with email reset code	Functional

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

FR-UC-29	Password reset code verification with expiry and attempt limit	Functional
FR-UC-30	Password reset execution with bcrypt hashing and reset-code cleanup	Functional
NFR-ST-01	Password storage security	Non-Functional
NFR-ST-02	JWT token security	Non-Functional
NFR-ST-13	Enumeration-safe forgot-password response	Non-Functional
NFR-ST-14	Secure reset code generation, expiry, and attempt limitation	Non-Functional
NFR-ST-15	Password reset password policy and password history protection	Non-Functional
CON-TECH-03	Supported file formats: PDF and TXT only	Constraint
CON-ENV-05	SMTP/Gmail credentials required for password reset email delivery	Constraint

2.3. Functional Requirements

Table 2.1. Authentication & User Management

ID	Name	Description	Priority
FR-UC-01	User Registration	The system shall allow Guest users to register with a valid email address and a password meeting minimum complexity requirements.	M
FR-UC-02	User Login & JWT Issuance	The system shall authenticate registered users via email and password and issue a signed JWT access token with a configurable expiry of 30 minutes.	M
FR-UC-03	JWT-Protected Endpoints	The system shall protect all non-public API endpoints with JWT Bearer token authentication and return HTTP 401 for unauthenticated requests.	M
FR-ST-04	Password Hashing	The system shall hash all user passwords using bcrypt before storage; plaintext passwords shall never be persisted in the database.	M
FR-ST-05	User Plan Types	The system shall support three user plan types - free, premium, and admin - with differentiated access to features based on plan type.	M
FR-ST-06	Analysis Quota Tracking	The system shall track the number of analyses performed by each user and enforce a monthly quota for free-tier users.	M
FR-UC-28	Forgot Password Request	The system shall accept a user's email via POST /auth/forgot-password, generate a 5-character case-sensitive alphanumeric reset code for registered users, store it with a 10-minute expiry and attempt count of 0 in the User record, send it to the user's email via the EmailService/Gmail SMTP, and return the same HTTP 200 response regardless of whether the email exists in the database.	M
FR-UC-29	Reset Code Verification	The system shall verify the submitted reset code via POST /auth/verify-reset-code by confirming that it matches the stored code, has not expired within the 10-minute TTL, and the reset attempt counter has not reached 5. Invalid codes shall return HTTP 400, expired codes shall return HTTP 400, and requests after the maximum attempt limit shall return HTTP 429.	M
FR-UC-30	Password Reset Execution	The system shall accept a new password via POST /auth/reset-password together with the email and valid reset code, validate the new password complexity, prevent reuse of the last 3 passwords through password history checking, hash the new password using bcrypt, update password_changed_at and password_history, persist the new password_hash, and clear reset_code, reset_code_expires_at, and reset_code_attempts.	M

Table 2.2. CV Upload & File Management

ID	Name	Description	Priority
FR-UC-07	CV File Upload	The system shall allow registered users to upload CV files in PDF or TXT format via a multipart HTTP request.	M
FR-UC-08	File Validation	The system shall validate the uploaded file's MIME type, extension, and size before accepting it. Files with mismatched MIME type shall be rejected with HTTP 415; files exceeding the size limit shall be rejected with HTTP 413.	M
FR-ST-09	Secure File Storage	The system shall store uploaded CV files with a UUID-generated filename to prevent path traversal attacks and filename collisions.	M

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

FR-ST-10	CV Record Creation	The system shall create a CV database record with status='pending' upon successful file upload and return the cv_id to the client within the same HTTP response.	M
FR-UC-11	CV List Retrieval	The system shall allow registered users to retrieve a list of all their uploaded CVs with associated status and metadata.	S
FR-UC-12	CV Deletion	The system shall allow registered users to delete a specific CV, which shall also cascade-delete the associated analysis result, suggestions, extracted skills, and career recommendations.	S

Table 2.3. CV Analysis Pipeline

ID	Name	Description	Priority
FR-SC-13	Asynchronous Analysis Trigger	The system shall trigger the CV analysis pipeline as a background task upon upload, without blocking the HTTP response to the client.	M
FR-DOC-14	Sequential Analysis Pipeline	The system shall execute the following analyzers in sequence: SectionDetector → SkillExtractor → ATSChecker → KeywordScorer → ExperienceEvaluator → ScoreCalculator → SuggestionGenerator.	M
FR-DOC-15	ATS Compliance Checking	The system shall evaluate at least 8 deterministic ATS rules including: contact information presence, email detection, content length sufficiency, absence of excessive capitalization, action verb usage, education section detection, experience section detection, and absence of table/image indicators.	M
FR-DOC-16	Weighted Score Calculation	The system shall compute the overall CV score using the following weighted formula: Completeness (25%) + Skills (25%) + ATS (20%) + Keywords (15%) + Experience (15%).	M
FR-DOC-17	Suggestion Generation	The system shall produce at least 3 actionable improvement suggestions categorized by type (content, skills, experience, ats, education) and assigned a priority level (high, medium, or low).	M
FR-SC-18	Pipeline Fault Tolerance	The system shall continue executing remaining analyzers if a single analyzer raises an exception; the error shall be logged and the pipeline shall not abort.	M
FR-UC-19	Analysis Result Retrieval	The system shall allow registered users to retrieve the analysis result for any of their uploaded CVs via a dedicated REST endpoint.	M

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Table 2.4. Career Recommendations & AI Summary

ID	Name	Description	Priority
FR-UC-20	Career Recommendation Generation	The system shall generate career recommendations for each analyzed CV by computing a match score between extracted keywords/skills and predefined role profiles.	M
FR-UC-21	Ranked Recommendation List	The system shall return the top-N career recommendations ranked by descending match score, where N is configurable.	S
FR-UC-22	AI-Generated CV Summary	The system shall allow premium users to request an AI-generated CV summary via the OpenAI GPT API. The summary shall be stored with the analysis result and returned on subsequent requests without re-calling the API.	M

Table 2.5. Subscription & Payment

ID	Name	Description	Priority
FR-ST-23	Payment Checkout Session	The system shall create a payment checkout session via the Stripe/Iyzico API when a registered user initiates a premium subscription upgrade.	M
FR-ST-24	Webhook Event Processing	The system shall process payment webhook events (checkout.session.completed, customer.subscription.deleted) and update the user's plan_type and subscription_end_at fields accordingly.	M
FR-ST-25	Webhook Signature Verification	The system shall verify Stripe webhook signatures before processing any event to prevent replay and spoofing attacks.	M

Table 2.6. Admin Functions

ID	Name	Description	Priority
FR-ST-26	Admin User Management	The system shall provide an admin-only endpoint for listing all users with their plan types, analysis counts, and registration dates.	M
FR-ST-27	Admin Action Logging	The system shall log all significant admin actions (user deletion, plan change, log inspection) to an AdminLog table with timestamp and actor information.	S

2.4. Non-Functional Requirements

2.4.1. Quality Requirements: Specific Quality Requirements

This subsection defines the **specific quality attributes** that the system must satisfy in addition to its functional behavior. These requirements describe how well the system performs its tasks, rather than what the system does.

Table 2.7. Security (Güvenlik)

ID	Name	Description	Priority
NFR-ST-01	Password Storage Security	The system shall hash all user passwords using bcrypt before storage; no plaintext password shall ever be stored in the database.	M
NFR-ST-02	JWT Token Security	The system shall sign all JWT tokens with a secret key and reject any request containing an expired or invalid token with HTTP 401.	M
NFR-ST-03	Secure File Naming	The system shall store all uploaded CV files using UUID-generated filenames to prevent path traversal attacks.	M
NFR-ST-04	Webhook Signature Validation	The system shall verify Stripe webhook signatures on every incoming payment event; unverified requests shall be rejected with HTTP 400.	M
NFR-ST-13	Security – Enumeration Safety	The /auth/forgot-password endpoint shall always return the same HTTP 200 response and message body regardless of whether the submitted email address is registered, preventing user enumeration attacks.	M
NFR-ST-14	Reset Code Security	The system shall generate password reset codes using a cryptographically secure random generator. Codes shall be 5-character case-sensitive alphanumeric values, expire after 10 minutes, and be protected by a maximum of 5 verification attempts.	M
NFR-ST-15	Password Reset Password Policy	The system shall require reset passwords to satisfy the same password complexity rules as registration and password change: minimum 8 characters, maximum 128 characters, at least one uppercase letter, one lowercase letter, and one number. The system shall prevent reuse of the last 3 passwords through password history checking.	M

Table 2.8. Reliability (Güvenilirlik)

ID	Name	Description	Priority
NFR-ST-05	Pipeline Fault Tolerance	The system shall continue executing remaining analysis steps if a single analyzer fails; no unhandled exception shall propagate to the user.	M
NFR-ST-06	Graceful Parsing Failure	The system shall update the CV status to 'failed' and return a descriptive error message if text extraction from the uploaded file produces no content.	M

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Table 2.9. Usability (Kullanılabilirlik).

ID	Name	Description	Priority
NFR-ST-07	Single-Page Results View	The system shall present the overall score, ATS breakdown, suggestions, extracted skills, and career recommendations on a single results page.	M
NFR-ST-08	Standardized Error Responses	The system shall return all error responses in a consistent JSON format containing error, message, and status fields.	S

Table 2.10. Availability (Erişebilirlik).

ID	Name	Description	Priority
NFR-ST-09	System Uptime	The system shall maintain an uptime of at least 99.5% in the production environment, excluding scheduled maintenance windows.	M

Table 2.11. Portability (Taşınabilirlik).

ID	Name	Description	Priority
NFR-ST-10	Container Deployment	The system shall be deployable via Docker containers; the frontend and backend shall run as independent containers.	M

Table 2.12. Maintainability (Bakımedilebilirlik).

ID	Name	Description	Priority
NFR-ST-11	Single Responsibility per Analyzer	Each analysis module shall implement the BaseAnalyzer interface and be responsible for exactly one analysis step.	M
NFR-ST-12	Database Migration Support	The system shall manage all database schema changes through Alembic migration scripts; no manual schema modification shall be required.	S

2.4.2. Quality Requirements: Performance Requirements

This subsection defines the **performance-related requirements** of the system. These requirements specify how fast, how often, and how much the system must handle during its operation.

Table 2.13. Timing/Synchronization (Zamanlama).

ID	Name	Description	Priority
NFR-SC-01	Analysis Pipeline Timing	The system shall complete the full CV analysis pipeline within 10 seconds for a standard 2-page PDF, excluding AI summary generation.	M

Table 2.14. Timing/Sampling Rate (Zamanlama/Örnekleme Oranı).

ID	Name	Description	Priority
NFR-SC-02	API Response Time	The system shall respond to all read-only REST API requests within 500 milliseconds at the 95th percentile under normal load.	M

Table 2.15. Processing Speed (İşleme Hızı).

ID	Name	Description	Priority
NFR-SC-03	Database Query Performance	The system shall retrieve a single analysis result with all related records from the database within 200 milliseconds.	M

Table 2.16. Data Volume / Storage (Veri Hacmi / Depo).

ID	Name	Description	Priority
NFR-SC-04	Analysis Result Retention	The system shall retain all analysis results, suggestions, extracted skills, and career recommendations in the database indefinitely until explicitly deleted by the user.	M

Table 2.17. Throughput / Data Transfer (Veri Akışı / Veri Aktarımı).

ID	Name	Description	Priority
NFR-SC-05	Concurrent Upload Support	The system shall support at least 50 simultaneous active CV upload sessions without degradation in response time.	M

2.4.3. Constraints

This subsection defines the **constraints** that restrict the design and implementation of the system. Constraints are conditions that must be satisfied and limit the possible solutions.

Table 2.18. Physical (Fiziksel).

ID	Name	Description	Priority
CON-TECH-01	Server Infrastructure	The system must be deployed on a server with a minimum of 2 GB RAM and 1 vCPU available for the backend container.	M
CON-TECH-02	File Storage Requirement	The system must maintain a dedicated local file storage directory (uploads/) accessible by the backend container at runtime.	M

Table 2.19. Interface (Arayüz-Kamera).

ID	Name	Description	Priority
CON-TECH-03	Supported File Formats	The system must accept only PDF and plain-text (TXT) file formats for CV upload.	M
CON-TECH-04	PDF Parsing Library	The system must use PyMuPDF (fitz) for PDF text extraction; scanned or image-only PDFs are not supported.	M
CON-TECH-05	REST API Communication	The frontend and backend must communicate exclusively via a RESTful JSON API over HTTP/HTTPS.	M

Table 2.20. Backend Framework Interface

ID	Name	Description	Priority
CON-TECH-06	Backend Framework	The system must be implemented using FastAPI (Python) as the backend framework.	M
CON-TECH-07	ORM and Migration Tool	The system must use SQLAlchemy as the ORM and Alembic for all database schema migrations.	M

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Table 2.21. Frontend Interface

ID	Name	Description	Priority
CON-TECH-08	Frontend Framework	The system must use React with Vite as the frontend framework.	M
CON-TECH-09	Rate Limiting	All API endpoints must be protected by rate limiting via slowapi; upload and analysis endpoints must be rate-limited per IP address.	M

Table 2.22. Environment (Ortam).

ID	Name	Description	Priority
CON-ENV-01	Database Environment	SQLite must be used only in development environments; PostgreSQL must be used in all production deployments.	M
CON-ENV-02	OpenAI API Dependency	The AI summary feature requires a valid OpenAI API key configured in the server environment.	M
CON-ENV-03	Payment Gateway Dependency	Subscription management requires a live Stripe API key and a registered webhook endpoint configured in the server environment.	M
CON-ENV-04	Container Deployment	The system must be deployable via Docker; the frontend and backend must run as independent containers.	M
CON-ENV-05	SMTP Email Dependency	The system shall require a valid Gmail account with an App Password configured as SMTP credentials (SMTP_HOST, SMTP_USERNAME, SMTP_PASSWORD environment variables) in the server environment.	M

Table 2.23. Design and Implementation (Tasarım ve Gerçekleştirme).

ID	Name	Description	Priority
CON-TECH-10	Analyzer Interface Compliance	Each analysis module must implement the BaseAnalyzer interface and be responsible for exactly one analysis step.	M
CON-TECH-11	UUID File Naming	All uploaded CV files must be stored with UUID-generated filenames; original filenames must not be used on the filesystem.	M

Table 2.24. Legal (Yasal).

ID	Name	Description	Priority
CON-REG-01	Data Privacy	All personal data including email addresses and CV content must be stored within the system boundary and must not be shared with external services other than OpenAI for AI summaries, the payment gateway for subscription processing, and Gmail SMTP for password reset email delivery.	M
CON-REG-02	Licensing Compliance	All third-party libraries and frameworks used in the system must be licensed under terms compatible with commercial use.	M

2.5. Assumptions

- Uploaded CV files are assumed to be digital (text-layer) PDFs. Scanned documents without a text layer are not supported and will produce an extraction error.
- Users are assumed to upload CVs written in English. Skill extraction, section detection, and keyword scoring dictionaries are English-language only in the current version.
- The OpenAI GPT API will be available and responsive during normal operating hours. AI summary generation may take an additional 5–15 seconds depending on API latency.
- The Stripe/Iyzico payment gateway will be accessible and its webhook events will be delivered reliably to the registered endpoint.
- The server environment will have a stable internet connection for outbound API calls to OpenAI and the payment gateway.
- The server infrastructure will have sufficient storage capacity to retain uploaded CV files and analysis results for all registered users.
- Free-tier users accept a monthly analysis quota as a commercially reasonable limitation of the platform.
- Registered users will have a modern web browser (Chrome, Firefox, Safari, Edge) to access the platform interface.
- Users (job seekers) will have basic digital literacy to upload a file and interpret the analysis results.
- The PostgreSQL database will be properly configured and accessible by the backend container in the production environment.
- Docker and Docker Compose will be available in the deployment environment for container orchestration.
- The JWT secret key and all API keys (OpenAI, Stripe) will be securely configured as environment variables before deployment.
- The number of concurrent users and upload sessions will remain within manageable limits for a single-server deployment during the initial release phase.
- The Gmail SMTP service (smtp.gmail.com:587) is assumed to be available and the configured Gmail App Password is assumed to remain valid. If SMTP settings are missing or the SMTP server is unavailable, password reset email delivery may fail; however, the forgot-password endpoint shall preserve the same generic response behavior and shall not reveal email existence or email delivery status to unauthenticated users. All other platform features continue to operate normally.

3. REQUIREMENTS ANALYSIS

3.1. Scenarios

Scenario 1 - Normal CV Upload and Analysis (Registered User)

1. A registered user logs into the CVision platform using their email and password. The system validates credentials and returns a JWT access token.
2. The user navigates to the Upload CV page and selects a PDF file (≤ 5 MB).
3. The frontend sends a multipart POST request to `/cvs/upload` with the JWT in the Authorization header.
4. The backend validates the token, checks the file's MIME type and size, generates a UUID filename, saves the file to disk, and creates a CV record with status='pending'.
5. The backend immediately returns HTTP 201 with the `cv_id`, then launches a background analysis task asynchronously.
6. The background task updates the CV status to 'processing', extracts text using PdfParser, and passes the text through the 7-step AnalysisEngine pipeline.
7. Upon completion, the analysis result is persisted to the database and the CV status is updated to 'completed'.
8. The user polls or navigates to the Results page. The frontend fetches GET `/analysis/{cv_id}` and displays the score, ATS report, suggestions, and career recommendations.

Scenario 2 - AI Summary Request (Premium User)

1. A premium user views their completed analysis result and clicks 'Generate AI Summary'.
2. The frontend sends a POST request to the AI summary endpoint with the `cv_id`.
3. The backend verifies the user's `plan_type` is 'premium'. If not, returns HTTP 403.
4. The backend constructs a prompt from the analysis context and calls the OpenAI GPT API.
5. The returned summary text is stored in the `AnalysisResult.ai_summary` field and returned to the frontend.
6. Subsequent requests for the same `cv_id` return the cached summary without re-calling the API.

Scenario 3 - Analysis Pipeline Failure (Invalid File / Scanned PDF)

1. A user uploads a scanned PDF (image-only, no text layer).
2. The backend saves the file and creates a CV record with status='pending'.
3. The background task calls `PdfParser.extract_text()`, which detects zero extractable text and raises `ValueError`.
4. The `AnalysisEngine` catches the parsing exception, updates the CV status to 'failed', and logs the error.
5. The user queries the analysis endpoint and receives a response indicating status='failed' with an appropriate error message.

Scenario 4 - Password Reset Flow (Registered User)

1. A user navigates to the Login page and clicks "Forgot Password".

2. The user is directed to /forgot-password and enters their email address.
3. The frontend sends POST /auth/forgot-password with the email.
4. The backend looks up the user by email. If the email belongs to a registered account, the system generates a 5-character case-sensitive alphanumeric reset code using a cryptographically secure random generator.
5. The backend stores the reset code in the User record, sets reset_code_expires_at to 10 minutes from the current time, and resets reset_code_attempts to 0.
6. The EmailService sends an HTML-formatted email via Gmail SMTP using SMTP/TLS on port 587. The email contains the reset code and informs the user that the code is valid for 10 minutes.
7. The system returns the same generic success message regardless of whether the email exists in the database, preventing user enumeration.
8. The user retrieves the code from their email and navigates to /reset-password.
9. The frontend sends POST /auth/verify-reset-code with the email and code.
10. The backend validates that the code exists, matches the submitted code, has not expired, and the reset attempt counter has not reached 5.
11. If the code is invalid, the system increments the attempt counter and returns HTTP 400. If the maximum attempt limit is reached, the system returns HTTP 429 and the user must request a new code.
12. After successful code verification, the user enters a new password.
13. The frontend sends POST /auth/reset-password with email, code, and new password.
14. The backend re-validates the code, validates password complexity, checks that the new password is not one of the last 3 passwords, hashes the new password with bcrypt, updates password_changed_at and password_history, and clears reset_code, reset_code_expires_at, and reset_code_attempts.
15. The user is redirected to the Login page and can sign in with the new password.

3.2. Use Case Models

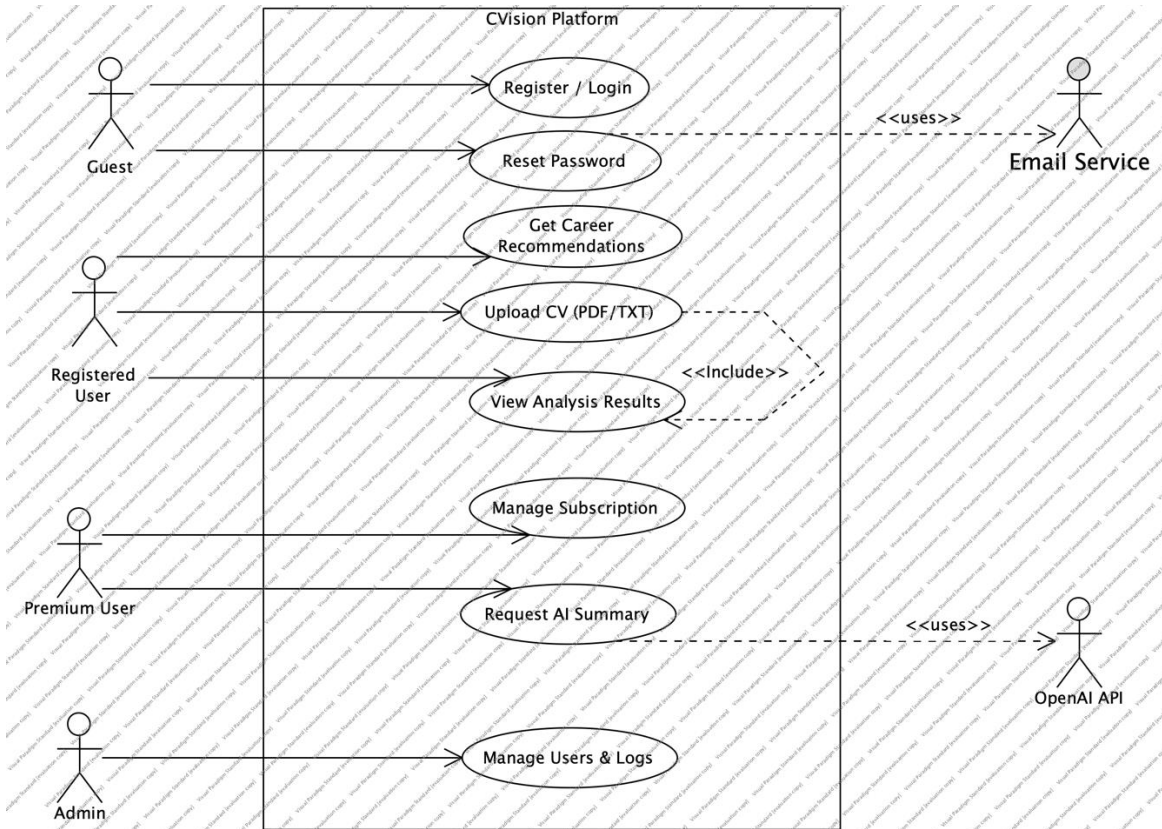


Figure 3.1. Use Case Diagram.

Use Case 1

Field	Description
Use Case Name	Upload CV
Actor(s)	Registered User
Description	The registered user uploads a CV file in PDF or TXT format to the CVision platform for automated analysis.
Preconditions	The user is logged in with a valid JWT token. The user has not exceeded their monthly analysis quota.
Main Flow	1. The user navigates to the Upload CV page. 2. The user selects a PDF or TXT file from their device. 3. The system validates the file's MIME type, extension, and size. 4. The system saves the file with a UUID-generated filename. 5. The system creates a CV record with status='pending' and returns the cv_id to the user. 6. The system triggers the analysis pipeline as a background task.
Alternative Flow	1. If the file MIME type is invalid, the system rejects the upload with HTTP 415. 2. If the file exceeds the size limit, the system rejects the upload with HTTP 413. 3. If the user has exceeded their monthly quota, the system returns HTTP 403.
Postconditions	The CV file is stored on the server, a CV record exists in the database with status='pending' or 'processing', and the analysis pipeline has been triggered.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Use Case 2

Field	Description
Use Case Name	View Analysis Results
Actor(s)	Registered User
Description	The registered user retrieves and views the automated analysis results for a previously uploaded CV.
Preconditions	The user is logged in with a valid JWT token. The user has previously uploaded a CV and its status is 'completed'.
Main Flow	1. The user navigates to the Results page for a specific CV. 2. The frontend sends GET /analysis/{cv_id} to the backend. 3. The system verifies the JWT token and confirms the CV belongs to the requesting user. 4. The system retrieves the analysis result from the database. 5. The system returns the overall score, ATS score, suggestions, extracted skills, and career recommendations. 6. The frontend displays all results on a single page.
Alternative Flow	1. If the CV status is 'processing', the system returns the current status and the user is asked to wait. 2. If the CV status is 'failed', the system returns an error message indicating the analysis could not be completed. 3. If the cv_id does not belong to the requesting user, the system returns HTTP 403.
Postconditions	The user has viewed their CV analysis results including score, ATS breakdown, suggestions, and career recommendations.

Use Case 3

Field	Description
Use Case Name	Request AI Summary
Actor(s)	Premium User, OpenAI API
Description	The premium user requests an AI-generated summary of their CV analysis via the OpenAI GPT API.
Preconditions	The user is logged in with a valid JWT token. The user's plan type is 'premium'. The CV analysis status is 'completed'.
Main Flow	1. The premium user clicks 'Generate AI Summary' on the results page. 2. The frontend sends a POST request to the AI summary endpoint with the cv_id. 3. The system verifies the user's plan type is 'premium'. 4. The system constructs a prompt from the analysis context and calls the OpenAI GPT API. 5. The system stores the returned summary in the AnalysisResult.ai_summary field. 6. The system returns the summary text to the frontend for display.
Alternative Flow	1. If the user's plan type is not 'premium', the system returns HTTP 403. 2. If the OpenAI API is unavailable, the system returns an error message and the ai_summary field remains null. 3. If a summary already exists for this CV, the system returns the cached summary without re-calling the API.
Postconditions	The AI-generated summary is stored in the database and displayed to the user on the results page.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Use Case 4

Field	Description
Use Case Name	Manage Users & Logs
Actor(s)	Admin
Description	The administrator manages registered users and inspects system activity logs via the admin dashboard.
Preconditions	The user is logged in with a valid JWT token and their plan type is 'admin'.
Main Flow	1. The admin navigates to the Admin Panel. 2. The system verifies the user's plan type is 'admin'. 3. The system retrieves and displays the full user list with plan types, analysis counts, and registration dates. 4. The admin selects a user to view details or change their plan type. 5. The system updates the user record and logs the action to the AdminLog table with timestamp and actor information.
Alternative Flow	1. If a non-admin user attempts to access the admin endpoint, the system returns HTTP 403. 2. If the target user does not exist, the system returns HTTP 404.
Postconditions	The selected user's information has been updated and the action has been recorded in the AdminLog table.

Use Case 5

Field	Description
Use Case Name	Reset Password
Actor(s)	Guest, Registered User, Email Service (Gmail SMTP)
Description	The user resets their account password by requesting a time-limited verification code sent to their registered email, verifying the code, and setting a new password that satisfies password complexity and password-history rules.
Preconditions	The user has a registered account. The user is not logged in. SMTP/Gmail credentials are configured and reachable in the server environment.
Main Flow	1. The user clicks "Forgot Password" on the Login page. 2. The user submits their email address at /forgot-password. 3. The frontend sends POST /auth/forgot-password to the backend. 4. The system returns a generic success message regardless of whether the email exists. 5. If the email belongs to a registered account, the system generates a 5-character case-sensitive alphanumeric reset code. 6. The system stores the reset code, 10-minute expiry timestamp, and attempt counter in the User record. 7. The EmailService sends an HTML password reset email through Gmail SMTP. 8. The user enters the code at /reset-password. 9. The frontend sends POST /auth/verify-reset-code. 10. The system verifies that the code matches, has not expired, and has not exceeded the 5-attempt limit. 11. The user enters a new password. 12. The system validates password complexity and checks password history. 13. The system hashes the new password, updates the User record, clears reset-code fields, and redirects the user to Login.
Alternative Flow	1. Wrong code: the system increments the reset attempt counter and returns HTTP 400. 2. Maximum attempts reached: after 5 attempts, the system returns HTTP 429 and requires a new reset code. 3. Expired code: after 10 minutes, the system returns HTTP 400 and the user must request a new code. 4. Unregistered email: the system returns the same generic success message and sends no email. 5. SMTP failure: the system logs the failure internally while preserving the generic forgot-password response.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

	6. Weak password: the system rejects the request with validation error. 7. Reused password: the system rejects the request because the new password matches one of the last 3 passwords.
Postconditions	The user's password_hash is updated with a new bcrypt hash, password_changed_at and password_history are updated, all reset-code fields are cleared, and the user can log in with the new password.

3.3. Data Models (Entity-Relationship (ER Diagrams))

The ER model represents the main data entities of the CVision platform and their relationships. A User owns multiple CVs, each CV produces a single AnalysisResult, and each AnalysisResult aggregates Suggestions, ExtractedSkills, and CareerRecommendations as child records.

Entities and Attributes:

User

- id (PK)
- email
- full_name
- password_hash
- tier
- role
- plan_type
- analysis_count
- quota_reset_at
- subscription_end_at
- stripe_customer_id
- reset_code (nullable)
- reset_code_expires_at (nullable)
- reset_code_attempts
- password_changed_at (nullable)
- password_history (nullable)
- created_at
- updated_at

CV

- id (PK)
- user_id (FK)
- filename

- status
- uploaded_at

AnalysisResult

- id (PK)
- cv_id (FK)
- overall_score
- ats_score
- ai_summary
- created_at

Suggestion

- id (PK)
- analysis_id (FK)
- category
- message

ExtractedSkill

- id (PK)
- analysis_id (FK)
- skill_name

CareerRecommendation

- id (PK)
- analysis_id (FK)
- role_title
- match_score

Relationships:

- User - CV: One User owns many CVs. Cardinality: 1 : N
- CV - AnalysisResult: One CV produces at most one AnalysisResult. Cardinality: 1 : 0..1
- AnalysisResult - Suggestion: One AnalysisResult contains many Suggestions. Cardinality: 1 : N
- AnalysisResult – ExtractedSkill: One AnalysisResult contains many ExtractedSkills. Cardinality: 1 : N
- AnalysisResult – CareerRecommendation: One AnalysisResult contains many CareerRecommendations. Cardinality: 1 : N

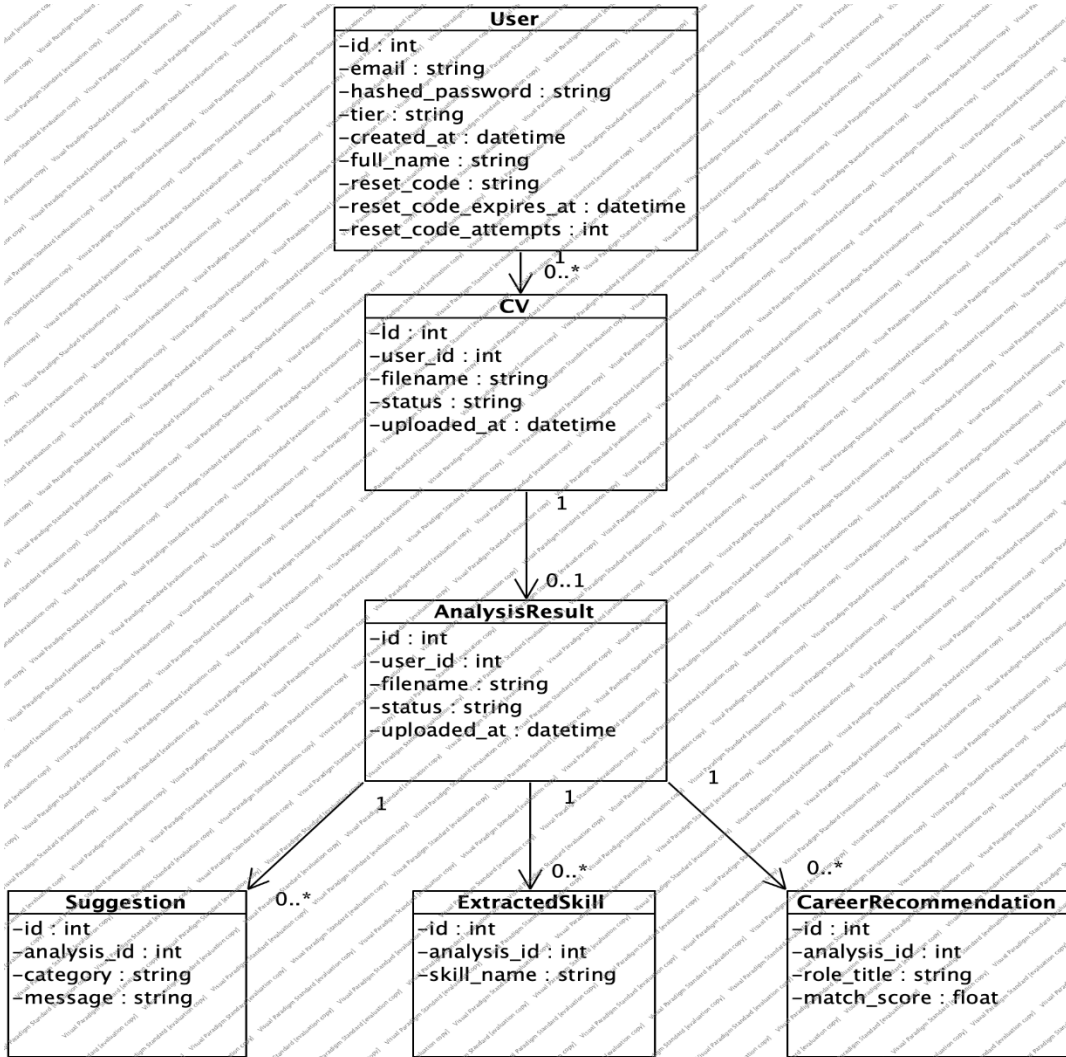


Figure 3.2. ER Data Model for the CVision platform.

This diagram represents the Entity-Relationship data model of the CVision platform using class diagram notation.

The purpose of this diagram is to show the main persistent data entities of the system, their attributes, and the relationships between them. The model defines how users, uploaded CV files, analysis results, suggestions, extracted skills, career recommendations, and password reset-related user data are stored in the database.

The main entities are User, CV, AnalysisResult, Suggestion, ExtractedSkill, and CareerRecommendation. A User can own multiple CV records. Each CV can produce at most one AnalysisResult. Each AnalysisResult can contain multiple Suggestions, ExtractedSkills, and CareerRecommendations.

The User entity also includes password reset fields such as reset_code, reset_code_expires_at, reset_code_attempts, password_changed_at, and password_history.

This diagram supports database design, backend implementation, requirement analysis, validation, and traceability by clarifying persistent system data and entity relationships.

3.4. Problem Domain Class/Object Models

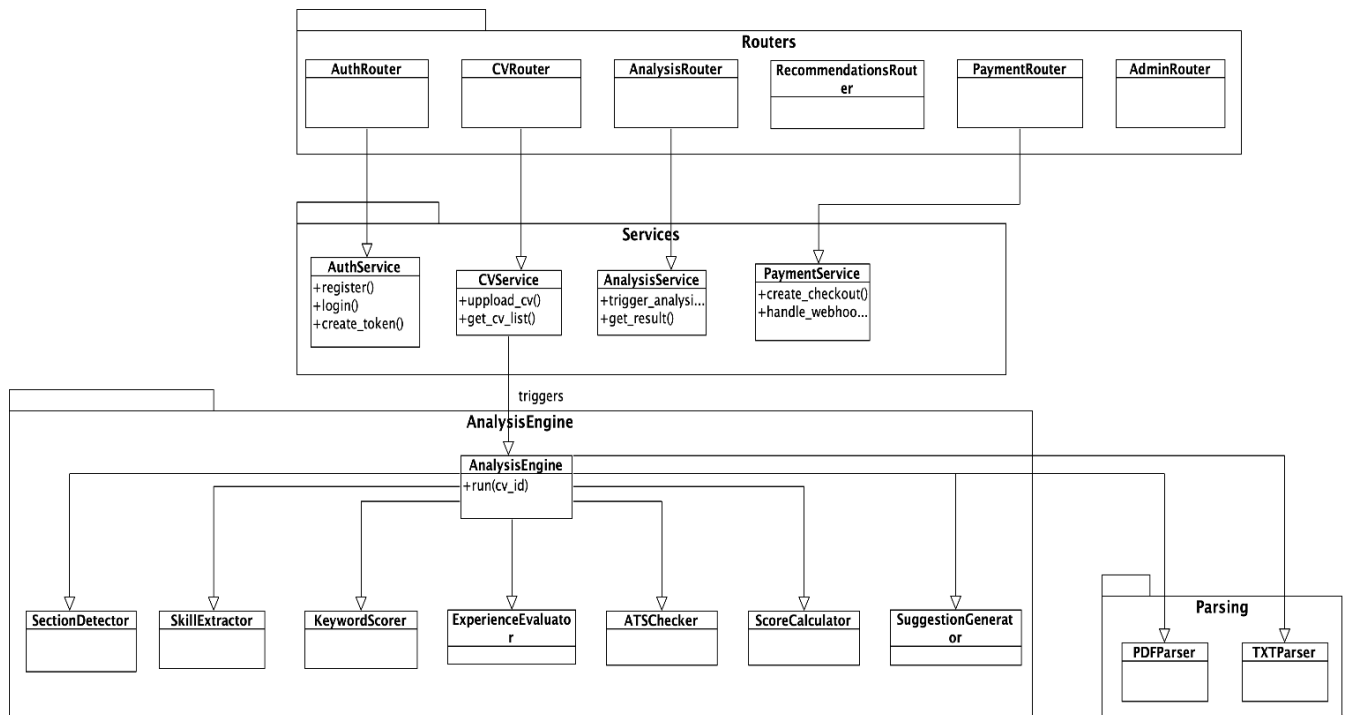


Figure 3.3 The main problem domain classes for the CVision platform.

This class diagram represents the main problem domain classes of the CVision platform. The model shows the key objects, their attributes, operations, and relationships. It helps describe the static structure of the system from the analysis perspective.

Main Classes and Operations:

AnalysisEngine

- `+run(cv_id)`
- Orchestrates the full 7-step analysis pipeline

CVService

- `+upload_cv()`
- `+get_cv_list()`
- Handles file operations and triggers the AnalysisEngine

AuthService

- `+register()`
- `+login()`
- `+create_token()`
- `+change_password()`
- `+forgot_password()`
- `+verify_reset_code()`
- `+reset_password()`

PaymentService

- +create_checkout()
- +handle_webhook()
- Manages subscription and payment gateway communication

EmailService

- +generate_reset_code()
- +get_reset_code_expiry()
- +send_reset_password_email()
- Generates secure 5-character case-sensitive alphanumeric password reset codes, calculates 10-minute expiry timestamps, prepares HTML password reset emails, and sends them via SMTP/Gmail using STARTTLS.

SectionDetector, SkillExtractor, KeywordScorer, ExperienceEvaluator, ATSChecker, ScoreCalculator, SuggestionGenerator

- Each implements a single analysis step
- All are invoked sequentially by AnalysisEngine

PDFParser, TXTParser

- Extract raw text from uploaded files
- Invoked by AnalysisEngine before analysis begins

Relationships:

- AuthRouter → AuthService: 1 : 1
- CVRouter → CVService: 1 : 1
- CVService → AnalysisEngine: triggers (1 : 1)
- AnalysisEngine → each Analyzer: 1 : 1
- AnalysisEngine → PDFParser / TXTParser: 1 : 1
- AuthService → EmailService: uses (1:1)
- AuthService / AuthRouter → EmailService: uses for reset-code generation and password reset email delivery (1 : 1)

3.5. Dynamic Models

Activity Diagram - CV Analysis Pipeline

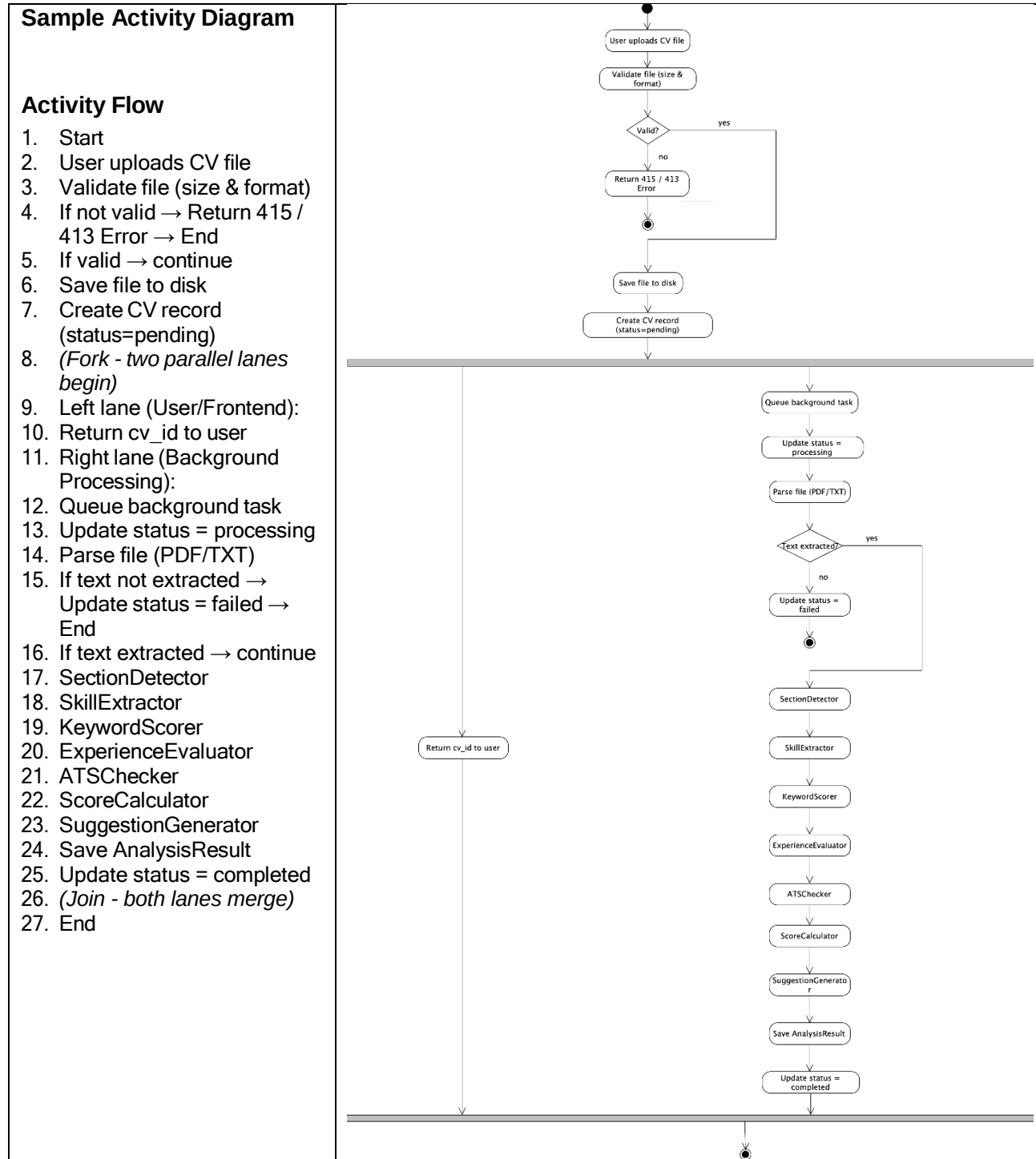


Figure 3.4. The activity diagram of the CV analysis pipeline.

This activity diagram shows the main workflow of the CV analysis system. The process starts with file upload and validation, continues with background text extraction and sequential analysis, and ends with the storage of the AnalysisResult and a status update to 'completed'. If validation or text extraction fails, the pipeline terminates early with an appropriate error status.

Sequence Diagram - CV Upload & Analysis Flow

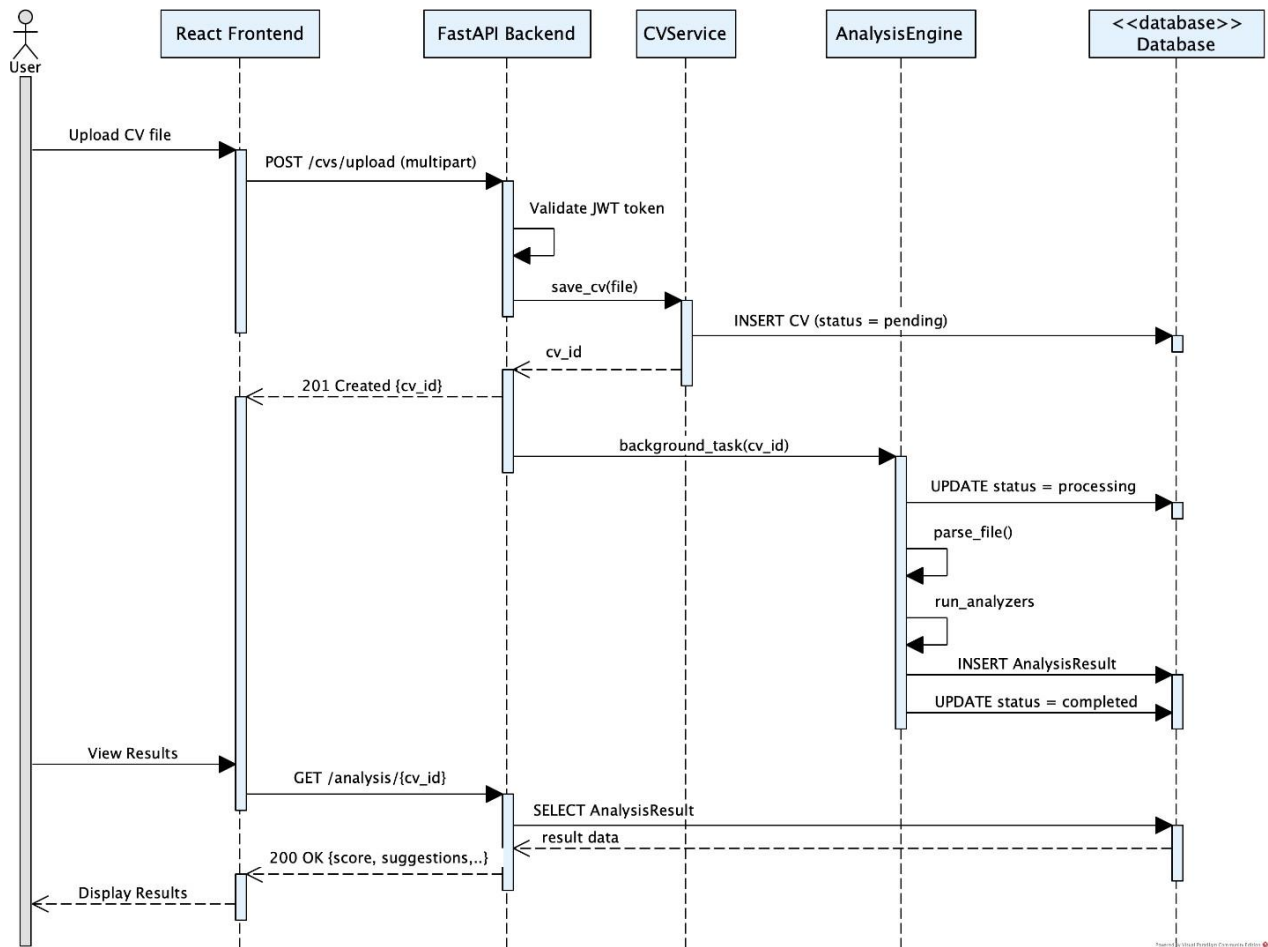


Figure 3.5. The sequence diagram of the CV upload and analysis flow.

This sequence diagram shows how the main system components interact during CV upload and analysis. The user uploads a file via the React Frontend, the FastAPI Backend validates the JWT token and delegates to CVService, and the AnalysisEngine runs asynchronously as a background task without blocking the HTTP response. The frontend retrieves the analysis results independently once processing is complete by calling GET /analysis/{cv_id}.

State Machine Diagram - CV Lifecycle

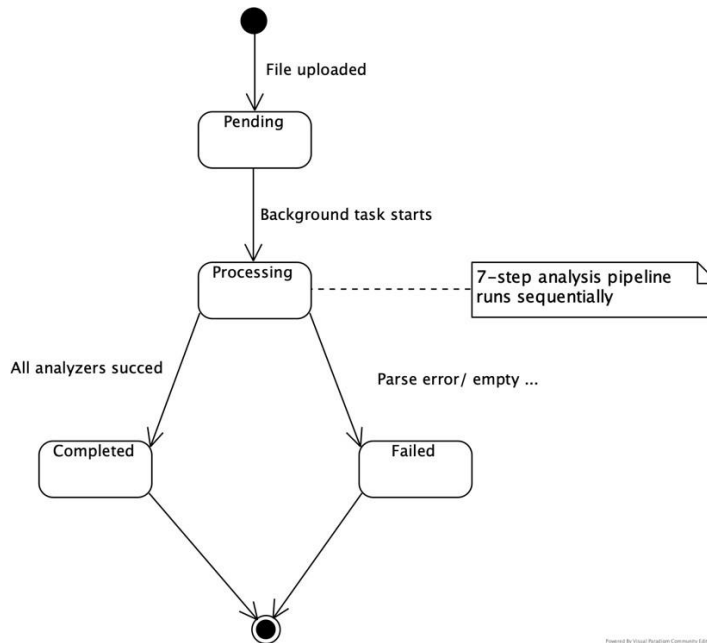


Figure 3.6. The state machine diagram of the CV lifecycle.

This state machine diagram shows all possible states of a CV document within the system. A CV starts in the Pending state after upload, transitions to Processing when the background task starts, and ends in either Completed (all analyzers succeed) or Failed (parse error or empty text). Both Completed and Failed are terminal states.

3.6. Component, Package and Deployment Models

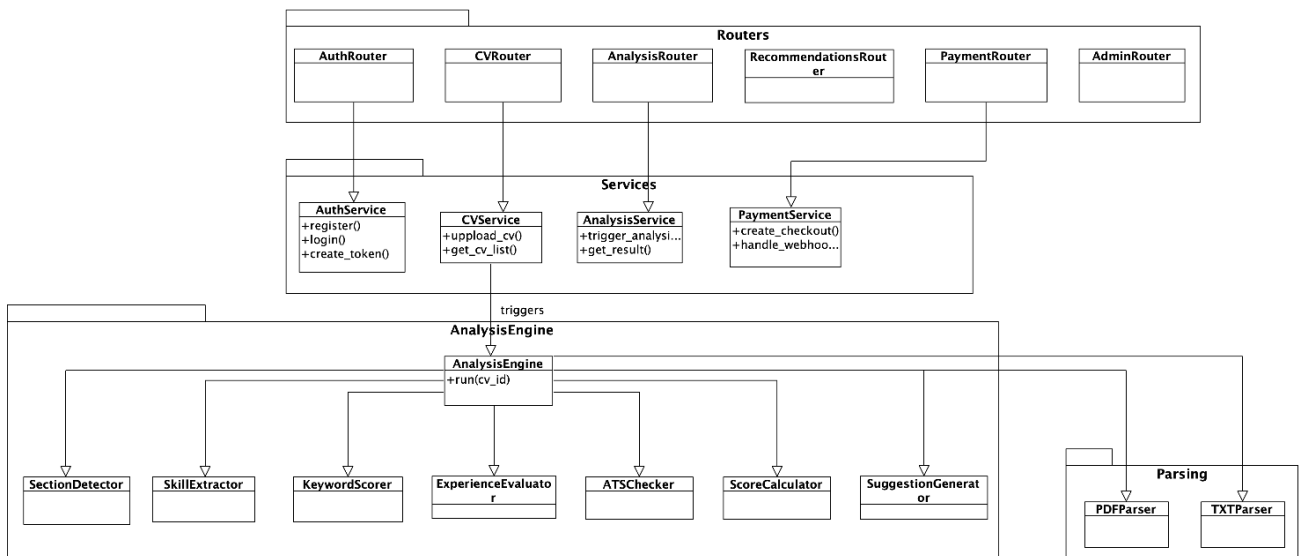


Figure 3.7. The component and package diagram of the CVision platform.

This component diagram shows the main software packages of the CVision backend and their dependencies. The Routers package handles all incoming HTTP requests and delegates to the Services package, which contains the business logic. The Services package triggers the Analysis Engine package, which sequentially executes all analyzer modules. The Parsing package provides file-format-specific text extraction utilities consumed by the Analysis Engine. The Authentication package also includes password recovery endpoints. AuthRouter handles forgot-password, verify-reset-code, and reset-password requests. It delegates reset-code generation, expiry calculation, and email delivery to EmailService. EmailService communicates with Gmail SMTP using STARTTLS and backend environment variables.

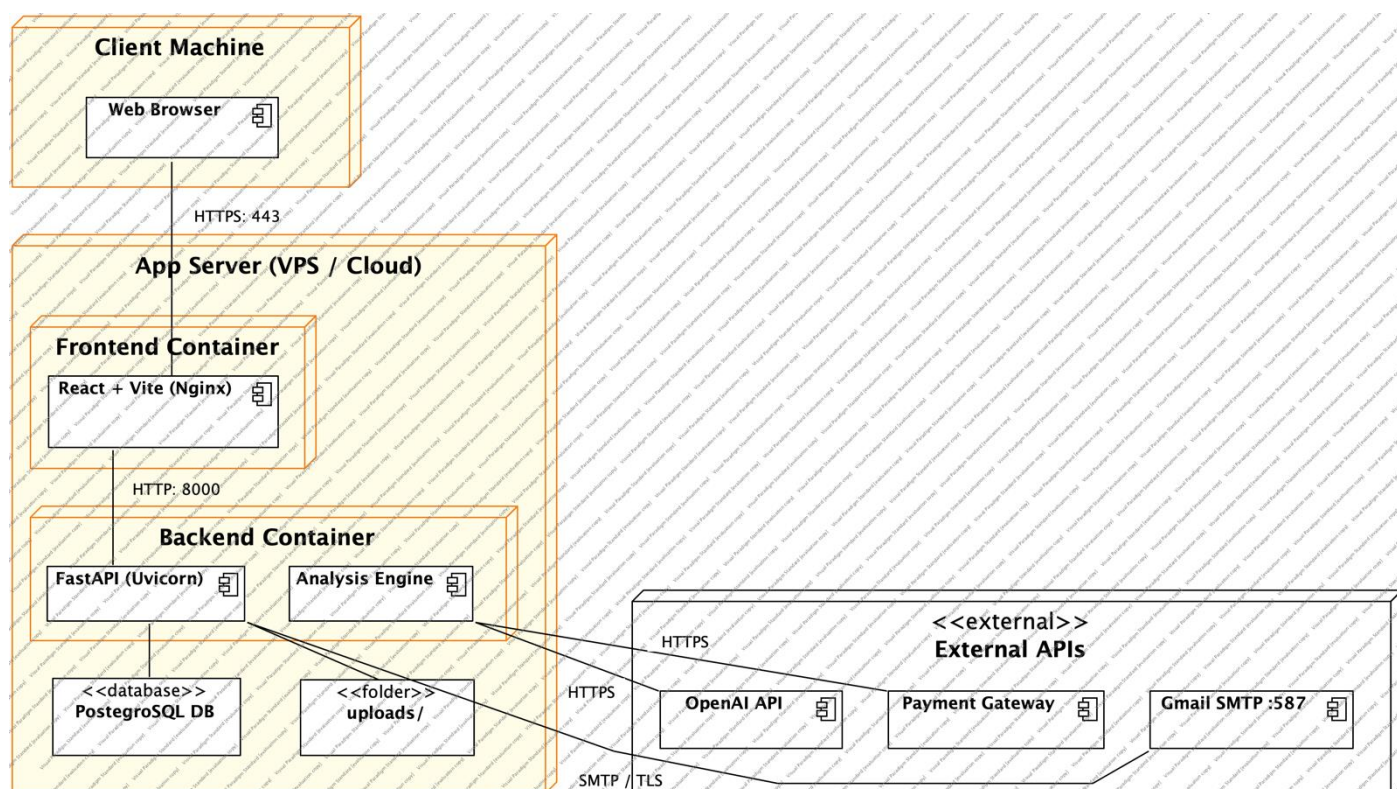


Figure 3.8. The deployment diagram of the CVision platform.

This deployment diagram shows how the CVision system is deployed across containerized infrastructure on a single VPS or cloud server. The Frontend Container runs React + Vite served by Nginx on port 443. The Backend Container runs FastAPI under Uvicorn on port 8000 and hosts the Analysis Engine in-process. Both containers share access to a PostgreSQL database and an uploads/ file storage directory. The backend communicates with external services – OpenAI API, Payment Gateway, and Gmail SMTP (port 587, STARTTLS) for transactional email delivery – via outbound HTTPS or SMTP/TLS. The Backend Container communicates with Gmail SMTP on port 587 using STARTTLS for password reset email delivery. SMTP credentials are supplied through backend environment variables and are not stored in source code.

3.7. System Architecture

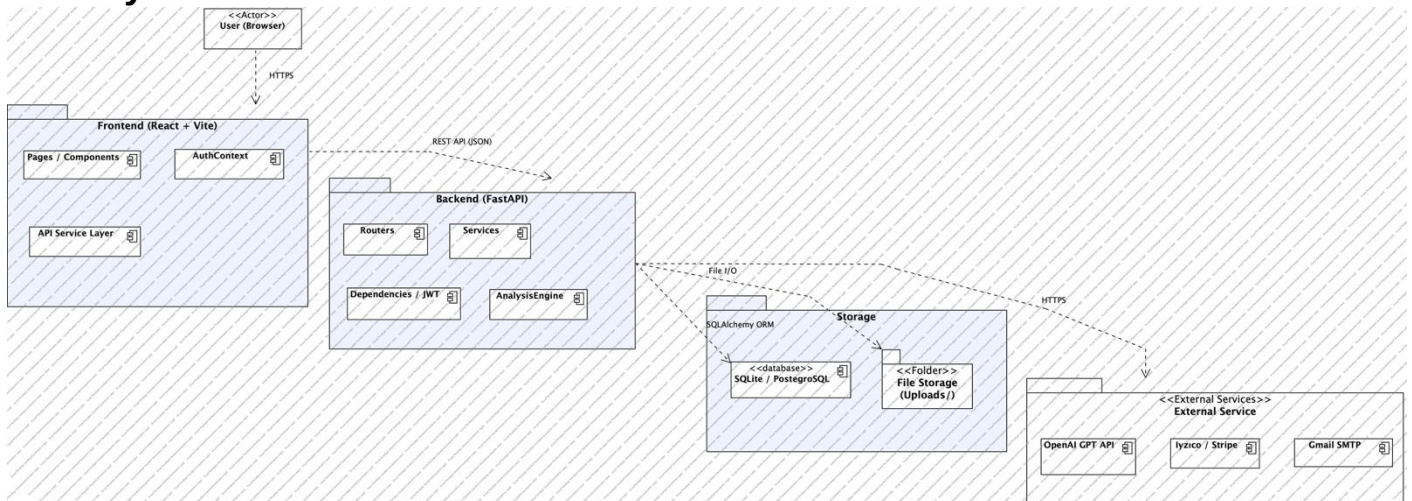


Figure 3.9. The system architecture diagram of the CVision platform.

This system architecture diagram presents the high-level structure of the CVision platform. The system is organized into four layers: the **Presentation Layer** (React + Vite frontend with Pages/Components, AuthContext, and API Service Layer), the **Application Layer** (FastAPI backend with Routers, Services, Dependencies/JWT, and Analysis Engine), the **Storage Layer** (SQLite/PostgreSQL database via SQLAlchemy ORM and local file storage), and the **External Services Layer** (OpenAI GPT API for AI summary generation, Iyzico/Stripe for payment processing, and Gmail SMTP for transactional password reset email delivery). The user accesses the system via HTTPS, the frontend communicates with the backend via REST API (JSON), and the backend communicates with storage via ORM and File I/O and with external services via outbound HTTPS. The Application Layer also includes EmailService, which is responsible for generating password reset codes, calculating reset-code expiry, preparing HTML reset emails, and sending them through the configured SMTP provider. The password reset workflow is exposed through AuthRouter endpoints and persists reset-code metadata in the User table.

3.8. User Interfaces

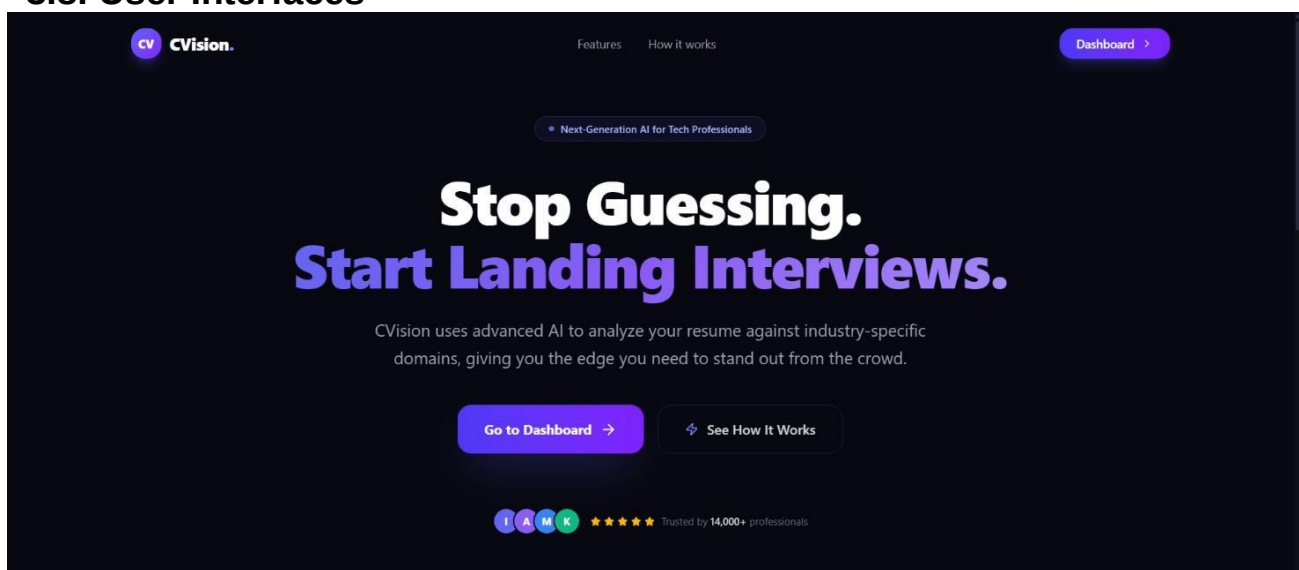


Figure 3.10. Landing page of the CVision platform.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

The purpose of this screen is to introduce the platform to new visitors. The screen displays the platform's key features, statistics, and a 3-step "How CVision Works" guide. The user can click the "Go to Dashboard" button to access the application or navigate to the Login and Register pages via the top menu.

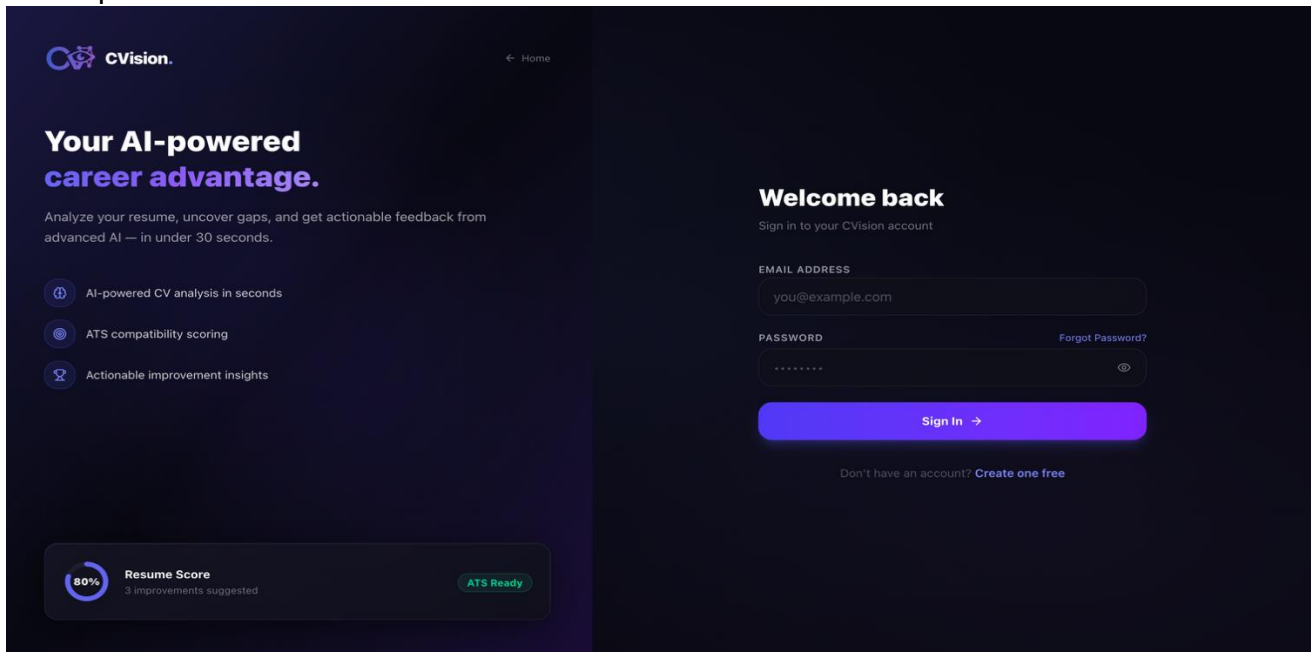


Figure 3.11. Login screen of the CVision platform.

The purpose of this screen is to authenticate existing users. The screen displays an email input field, a password input field, and a Sign In button. The user enters their credentials and clicks Sign In; on success, they are redirected to the Dashboard. The Login screen also includes a "Forgot Password?" link. When clicked, the user is redirected to the Forgot Password page to start the password reset workflow.

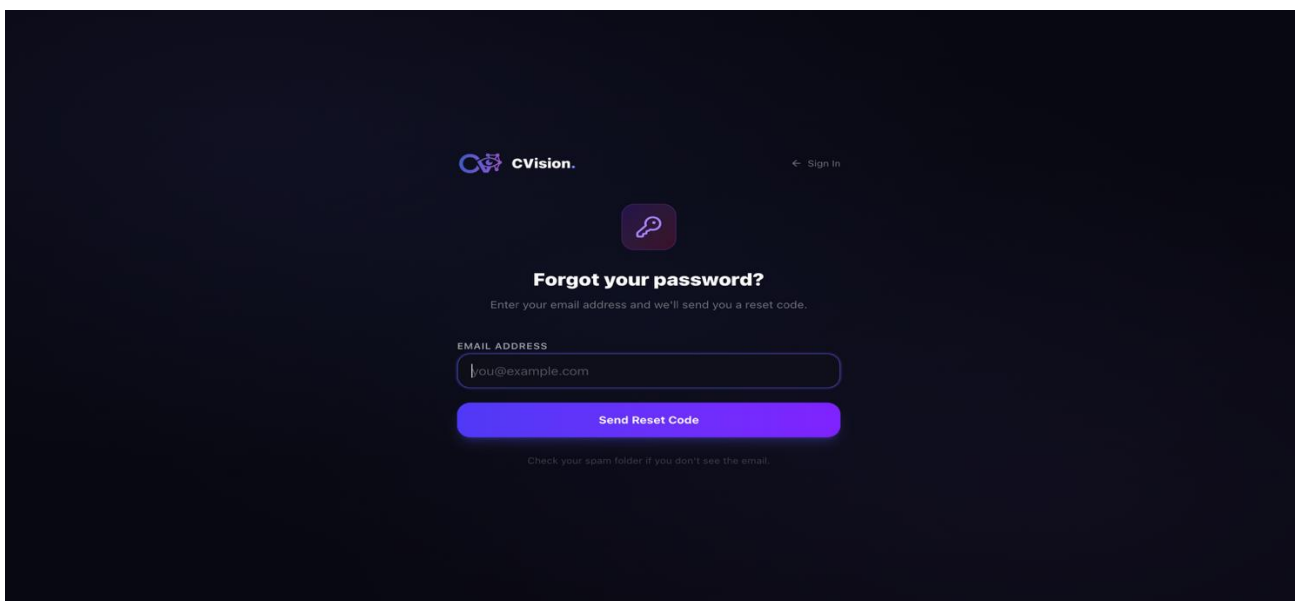


Figure 3.12. Forgot Password screen of the CVision platform.

The purpose of this screen is to allow users to start the password recovery process when they forget their account password. The screen contains an email input field and a “Send Reset Code” button. The user enters their registered email address, and the frontend sends a forgot-password request to the backend. If the email belongs to a registered account, the system generates a 5-character case-sensitive reset code and sends it to the user via Gmail SMTP. For security reasons, the system displays a generic response regardless of whether the submitted email address exists.

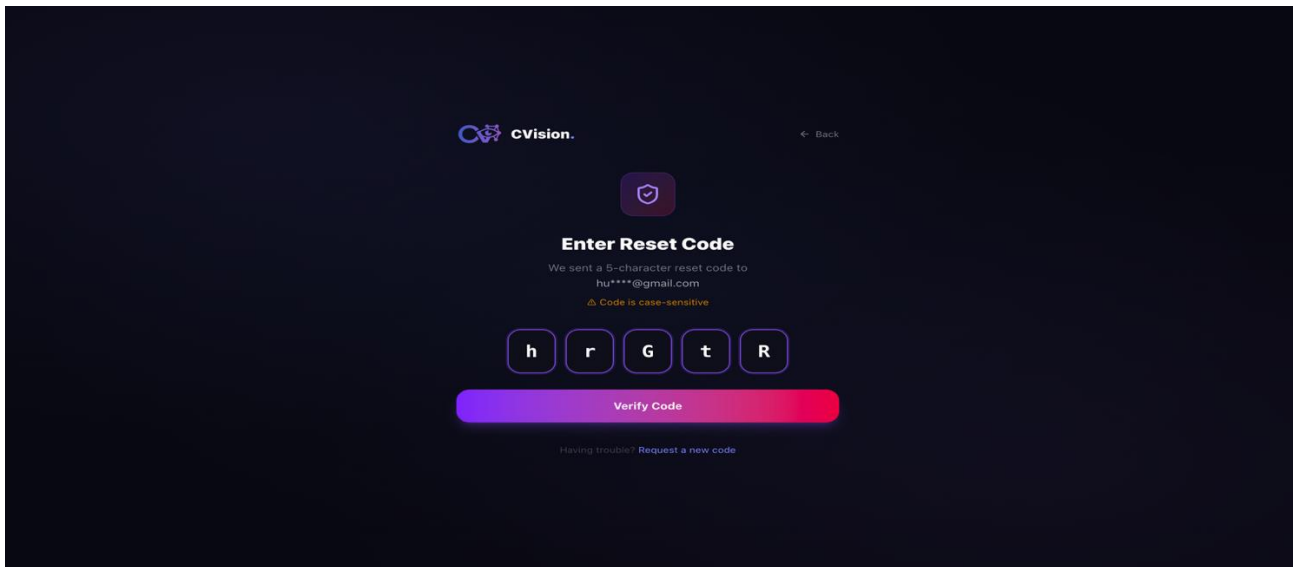


Figure 3.13. Reset Code Verification screen of the CVision platform.

The purpose of this screen is to verify the reset code sent to the user’s email address. The screen informs the user that a 5-character reset code has been sent and clearly states that the code is case-sensitive. The user enters the code into five separate input boxes and clicks the “Verify Code” button. The backend checks whether the submitted code matches the stored reset code, has not expired within the 10-minute validity period, and has not exceeded the maximum attempt limit.

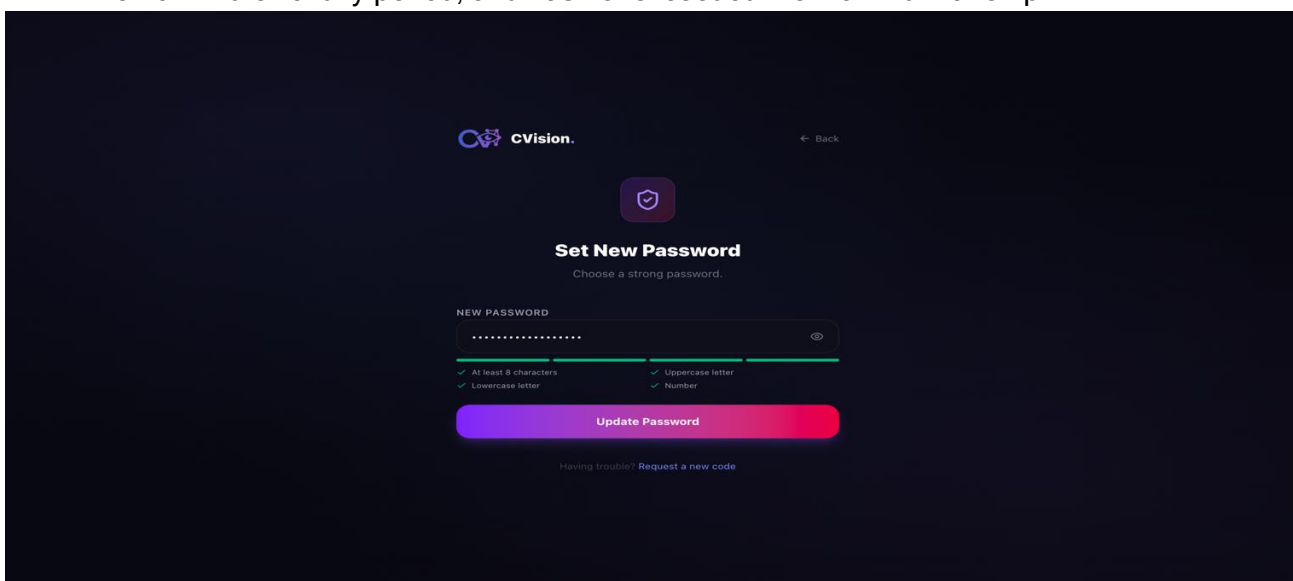


Figure 3.14. Set New Password screen of the CVision platform.

The purpose of this screen is to allow the user to create a new password after successfully verifying the reset code. The screen contains a password input field, a password visibility toggle, and real-

time password validation indicators. The password must satisfy the required complexity rules, including at least 8 characters, one uppercase letter, one lowercase letter, and one number. When the user clicks the “Update Password” button, the backend validates the reset code again, checks password complexity and password history, hashes the new password with bcrypt, updates the user record, and clears the reset-code fields.

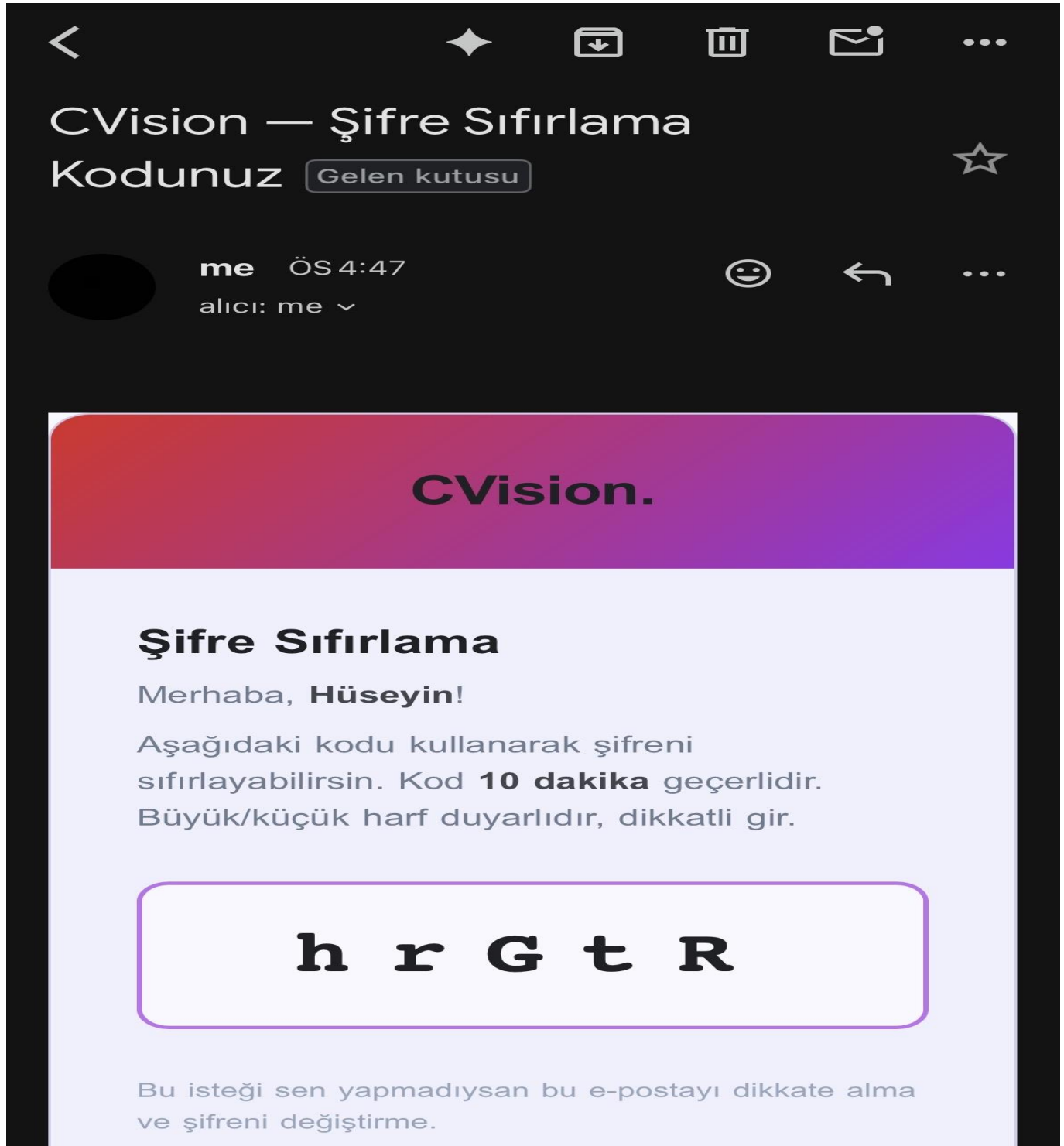
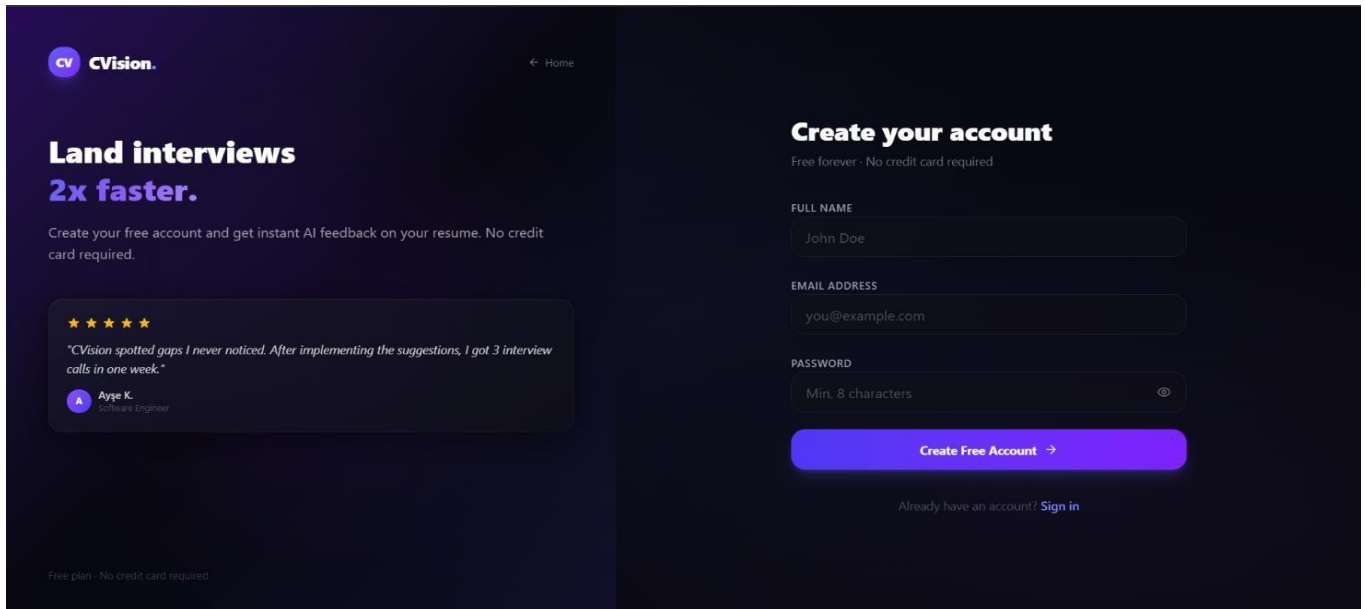


Figure 3.15. Password Reset Email received through Gmail SMTP.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

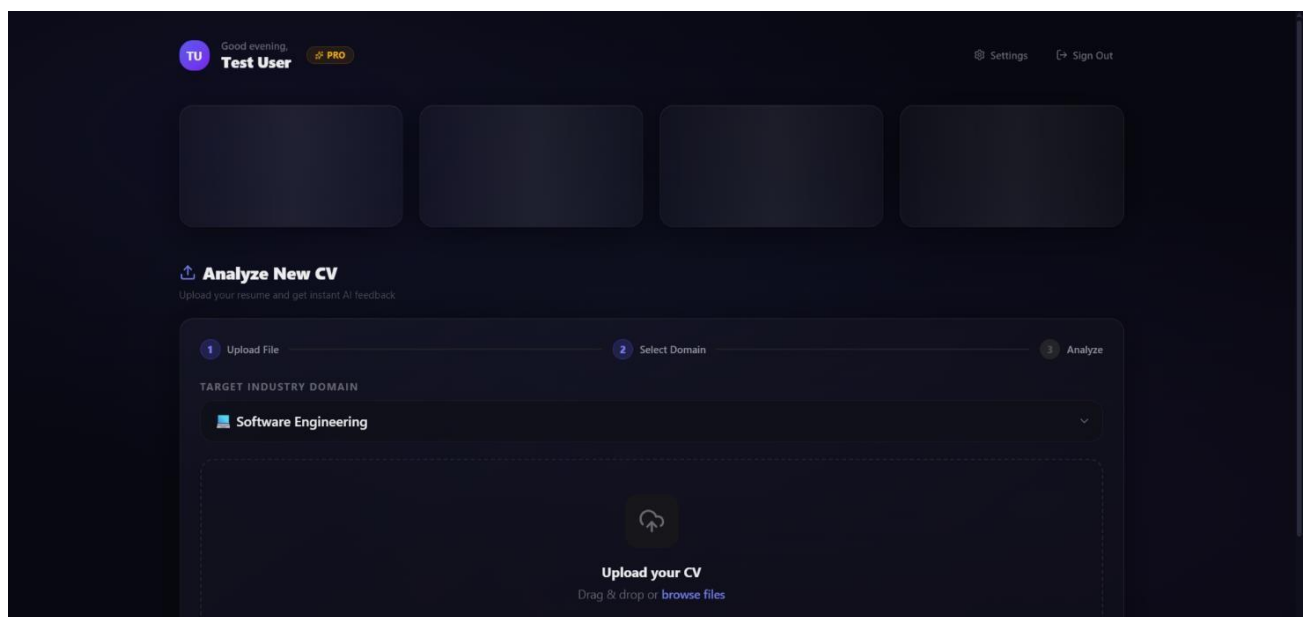
This screen shows the password reset email delivered to the user through Gmail SMTP. The email contains the CVision branding, a short explanation of the password reset process, the 5-character case-sensitive reset code, and a warning that the code is valid for 10 minutes. This confirms the email delivery step of the password reset workflow and demonstrates the integration between the backend EmailService and Gmail SMTP.



The registration screen features a dark blue background. On the left, a sidebar contains the CVision logo, a 'Home' link, and a section titled 'Land interviews 2x faster.' with a testimonial from Ayge K., a Software Engineer. The main area is titled 'Create your account' with the subtext 'Free forever - No credit card required'. It includes three input fields: 'FULL NAME' (containing 'John Doe'), 'EMAIL ADDRESS' (containing 'you@example.com'), and 'PASSWORD' (with a hint 'Min. 8 characters' and an eye icon). A prominent blue 'Create Free Account' button is at the bottom, followed by a link for users who 'Already have an account? Sign in'.

Figure 3.16. Registration screen of the CVision platform

The purpose of this screen is to allow new users to create a free account. The screen displays a full name field, an email field, a password field, and a Create Free Account button. The user fills in the form and submits; on success, the account is created and the user is redirected to the Dashboard.



The dashboard screen has a dark blue background. At the top, a user profile for 'Test User' is shown with a 'PRO' badge, a 'Good evening.' greeting, and links for 'Settings' and 'Sign Out'. Below this is a row of four empty rectangular cards. The main section is titled 'Analyze New CV' with the instruction 'Upload your resume and get instant AI feedback'. It features a three-step process: '1 Upload File', '2 Select Domain', and '3 Analyze'. Under '2 Select Domain', a dropdown menu shows 'Software Engineering' as the selected 'TARGET INDUSTRY DOMAIN'. Below the dropdown is a large dashed box with a cloud upload icon and the text 'Upload your CV' and 'Drag & drop or browse files'.

Figure 3.17. Dashboard and CV upload screen of the CVision platform.

The purpose of this screen is to allow authenticated users to upload a CV and start analysis. The screen displays a 3-step progress bar (Upload File → Select Domain → Analyze), a Target Industry Domain dropdown menu, and a drag-and-drop file upload area. The user selects a domain, uploads their CV file, and proceeds to trigger the analysis.

3.9. Minimum Requirements for This Project

- 4 Scenarios, including password reset flow
- 1 Use Case Diagram
- 5 Use Case Descriptions, including Reset Password
- 1 Data Model updated with password reset fields
- Dynamic models for CV analysis flow, including activity, sequence, and state machine diagrams.
- 1 Architecture Diagram updated with Gmail SMTP / EmailService
- 8 UI Screens, including Landing, Login, Forgot Password, Reset Code Verification, Set New Password, Password Reset Email, Registration, and Dashboard/CV Upload screens.

4. REQUIREMENTS VALIDATION

4.1. Validation Approach

1. Correctness

- The requirements accurately reflect the expected behavior of the CVision platform.
- Functional requirements (e.g., CV upload, 7-step analysis pipeline, ATS checking) match the system goals defined in the use case diagram and source code.
- ScoreCalculator weights sum to 1.0; ATS checks are deterministic and verifiable.

2. Completeness

- All major system functionalities are covered:
 1. User registration and authentication
 2. CV file upload and validation
 3. Automated analysis pipeline (7 steps)
 4. ATS compliance checking
 5. Career recommendations
 6. AI summary generation (Premium)
 7. Subscription and payment management
 8. Admin user and log management
 9. Password reset via email verification code, including forgot-password request, Gmail SMTP email delivery, reset-code verification, password complexity validation, password history checking, and reset-code cleanup
- Both normal operation and error scenarios (e.g., scanned PDF, analyzer failure, invalid webhook) are considered.

3. Consistency

- No conflicting requirements were identified.
- Terminology (e.g., CV, AnalysisResult, plan_type, status, cv_id) is used consistently throughout the document.
- All requirement IDs are unique and not reused. Password reset requirements are defined under FR-UC-28, FR-UC-29, and FR-UC-30, while reset-related non-functional requirements use separate IDs such as NFR-ST-13, NFR-ST-14, and NFR-ST-15.

4. **Clarity**

- Each requirement contains a single, well-defined system behavior.
- Ambiguous terms are avoided or replaced with measurable values (e.g., " ≤ 10 seconds", " ≥ 8 ATS checks", "at least 3 suggestions").
- All functional requirements start with "The system shall...".

5. **Realism**

- All requirements are feasible using the selected technology stack (FastAPI, PyMuPDF, OpenAI API, Stripe).
- Performance thresholds (e.g., ≤ 10 s pipeline, ≤ 500 ms API response) are consistent with FastAPI benchmark data and typical cloud server capabilities.

6. **Traceability**

- Each requirement is uniquely identified (e.g., FR-UC-01, NFR-ST-01, CON-TECH-01).
- System behaviors can be traced to specific functional requirements.
- Requirements are consistent with:
 1. Use Case Models (Section 3.2)
 2. System Architecture (Section 3.7)
 3. Data Models (Section 3.3)

4.2. Acceptance Criteria

Requirement ID	Acceptance Criteria
FR-UC-01	A new user registers with a valid email and password of ≥ 8 characters. Duplicate email returns HTTP 409.
FR-UC-02	A valid JWT token is returned on successful login. Invalid credentials return HTTP 401.
FR-UC-03	Protected endpoints without a valid JWT return HTTP 401.
FR-UC-07	A valid PDF file ($\leq$ size limit) is uploaded successfully and server returns HTTP 201 with a numeric cv_id within 2 seconds.
FR-UC-08	A file with mismatched MIME type returns HTTP 415. A file exceeding the size limit returns HTTP 413.
FR-SC-13	HTTP 201 is returned to the client before the analysis pipeline completes.
FR-DOC-14	All 7 analyzers run in the correct sequence and a non-null AnalysisResult is stored with overall_score > 0 .
FR-DOC-15	ATSChecker returns results for at least 8 named checks, each with a pass/fail status and a message.
FR-DOC-16	overall_score is in range [0, 100]. Completeness (25%) + Skills (25%) + ATS (20%) + Keywords (15%) + Experience (15%) = 100%.
FR-DOC-17	At least 3 suggestions are generated, each with a non-empty category, priority, and message field.
FR-SC-18	If one analyzer raises an exception, the remaining analyzers continue. The final AnalysisResult is still stored.
FR-UC-22	A free-tier user requesting AI summary receives HTTP 403. A premium user receives a non-empty ai_summary string.
FR-ST-23	A premium upgrade request creates a valid checkout session URL.
FR-ST-24	A valid payment webhook updates the user's subscription information.
FR-ST-25	A webhook request with an invalid Stripe signature returns HTTP 400 without modifying any user plan data.
FR-ST-26	Admin users can list users; non-admin users receive HTTP 403.
FR-ST-27	When an admin performs a significant action, the system creates an AdminLog record with actor information and timestamp.
NFR-SC-01	Analysis pipeline for a standard 2-page English CV completes in ≤ 10 seconds on production hardware.
NFR-ST-01	Database inspection confirms zero plaintext passwords. All hashes begin with '2b' (bcrypt prefix).
CON-REG-02	All third-party libraries pass a license compatibility check (MIT, Apache 2.0, or equivalent).
FR-UC-28	A forgot password request via POST /auth/forgot-password returns HTTP 200 with a generic message regardless of whether the email exists. For registered emails, a 5-character case-sensitive alphanumeric reset code is saved in the database with a 10-minute expiry, reset_code_attempts is set to 0, and email delivery is attempted via Gmail SMTP.
FR-UC-29	POST /auth/verify-reset-code returns HTTP 200 {valid: true} when a correct, non-expired code is submitted. Invalid codes return HTTP 400, expired codes return HTTP 400, and requests after 5 attempts return HTTP 429 or require a new reset code.
FR-UC-30	POST /auth/reset-password verifies the code one final time, validates password complexity and password history, hashes the new password with bcrypt, persists it, updates password_changed_at and password_history, clears reset_code, reset_code_expires_at, and reset_code_attempts, and returns HTTP 200. Subsequent login with the new password succeeds.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

NFR-ST-13	The /auth/forgot-password endpoint always returns the same HTTP 200 response and message body regardless of whether the submitted email address is registered, preventing user enumeration attacks.
NFR-ST-14	Reset codes are generated using a cryptographically secure random generator, are 5-character case-sensitive alphanumeric values, expire after 10 minutes, and are protected by a maximum of 5 verification attempts.
NFR-ST-15	Reject weak reset passwords and passwords reused from the last 3 stored hashes.

5. TRACEABILITY

ID	Name	Use Case	Component / Module	Test Case
FR-UC-01	User Registration	UC-01 Register / Login	AuthService / AuthRouter	Register with valid email; verify HTTP 201 and bcrypt hash stored
FR-UC-02	User Login & JWT Issuance	UC-01 Register / Login	AuthService / JWT Dependency	Login with valid credentials; verify JWT returned
FR-UC-03	JWT-Protected Endpoints	UC-01 Register / Login	JWT Dependency / FastAPI Depends()	Call protected endpoint without token; verify HTTP 401
FR-ST-04	Password Hashing	UC-01 Register / Login	AuthService / bcrypt	Inspect DB; verify all hashes begin with '
FR-ST-05	User Plan Types	UC-06 Manage Subscription	AuthService / User Model	Verify plan_type field accepts free, premium, admin only
FR-ST-06	Analysis Quota Tracking	UC-02 Upload CV	CVService / User Model	Exceed free quota; verify HTTP 403 returned
FR-UC-07	CV File Upload	UC-02 Upload CV	CVRouter / CVService	Upload valid PDF; verify HTTP 201 and cv_id returned
FR-UC-08	File Validation	UC-02 Upload CV	CVService.validate_file()	Upload valid PDF; verify HTTP 201 and cv_id returned
FR-ST-09	Secure File Storage	UC-02 Upload CV	CVService / File System	Verify stored filename is UUID format, not original filename
FR-ST-10	CV Record Creation	UC-02 Upload CV	CVService / Database	Verify CV record created with status='pending' and cv_id returned in same response
FR-UC-11	CV List Retrieval	UC-02 Upload CV	CVRouter / CVService	Verify authenticated user receives list of their CVs with status and metadata

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

FR-UC-12	CV Deletion	UC-02 Upload CV	CVRouter / CVService	Delete CV; verify cascade deletion of AnalysisResult, Suggestions, Skills, Recommendations
FR-SC-13	Asynchronous Analysis Trigger	UC-02 Upload CV	CVRouter / BackgroundTasks	Verify HTTP 201 is returned before analysis pipeline completes
FR-DOC-14	Sequential Analysis Pipeline	UC-03 View Analysis Results	AnalysisEngine	Verify all 7 analyzers execute in correct order; AnalysisResult stored
FR-DOC-15	ATS Compliance Checking	UC-03 View Analysis Results	ATSChecker	Verify at least 8 named checks returned, each with pass/fail status
FR-DOC-16	Weighted Score Calculation	UC-03 View Analysis Results	ScoreCalculator	Verify overall_score in [0,100]; weights sum to 1.0
FR-DOC-17	Suggestion Generation	UC-03 View Analysis Results	SuggestionGenerator	Verify at least 3 suggestions returned with category, priority, and message
FR-SC-18	Pipeline Fault Tolerance	UC-03 View Analysis Results	AnalysisEngine	Inject exception into one analyzer; verify remaining analyzers continue
FR-UC-19	Analysis Result Retrieval	UC-03 View Analysis Results	AnalysisRouter / AnalysisService	Authenticated user fetches GET /analysis/{cv_id}; verify 200 OK with full result
FR-UC-20	Career Recommendation Generation	UC-04 Get Career Recommendations	RecommendationsRouter / KeywordScorer	Verify at least 3 role recommendations returned with match_score > 0
FR-UC-21	Ranked Recommendation List	UC-04 Get Career Recommendations	RecommendationsRouter	Verify recommendations returned in descending match_score order
FR-UC-22	AI-Generated CV Summary	UC-05 Request AI Summary	AnalysisService / OpenAI Client	Free user: verify HTTP 403. Premium user: verify non-empty ai_summary returned
FR-ST-23	Payment Checkout Session	UC-06 Manage Subscription	PaymentService / Stripe API	Verify checkout session URL returned on upgrade request
FR-ST-24	Webhook Event Processing	UC-06 Manage Subscription	PaymentRouter / PaymentService	Send valid webhook event; verify plan_type updated to 'premium'
FR-ST-25	Webhook Signature Verification	UC-06 Manage Subscription	PaymentRouter / Stripe SDK	Send webhook with invalid signature; verify HTTP 400 and no plan change
FR-ST-26	Admin User Management	UC-07 Manage Users & Logs	AdminRouter / AdminService	Non-admin user: verify HTTP 403. Admin: verify full user list returned
FR-ST-27	Admin Action Logging	UC-07 Manage Users & Logs	AdminService / AdminLog Model	Perform admin action; verify AdminLog entry created with timestamp
FR-UC-28	Forgot Password Request	UC-08 Reset Password	AuthRouter / EmailService / User Model	Submit registered email; verify generic HTTP 200 response, reset_code saved, expiry set to 10 minutes, attempt counter reset to 0, and email send attempted via SMTP. Submit unregistered email; verify same generic HTTP 200 response.
FR-UC-29	Reset Code Verification	UC-08 Reset Password	AuthRouter / User Model	Submit valid code; verify HTTP 200 {valid: true}. Submit invalid code; verify HTTP 400. Submit expired code; verify HTTP 400. Exceed 5 attempts; verify HTTP 429 or new-code requirement.

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

FR-UC-30	Password Reset Execution	UC-08 Reset Password	AuthRouter / bcrypt / User Model	Submit valid code and new password; verify password_hash update, reset-code cleanup, and successful login.
-----------------	--------------------------	----------------------	----------------------------------	--

Non-Functional Requirements

ID	Name	Use Case	Component / Module	Test Case
NFR-ST-01	Password Storage Security	UC-01 Register / Login	AuthService / bcrypt	Verify zero plaintext passwords in DB; all hashes start with '
NFR-ST-02	JWT Token Security	UC-01 Register / Login	JWT Dependency	Send expired token; verify HTTP 401 returned
NFR-ST-03	Secure File Naming	UC-02 Upload CV	CVService / File System	Verify all stored filenames are UUID format
NFR-ST-04	Webhook Signature Validation	UC-06 Manage Subscription	PaymentRouter / Stripe SDK	Send tampered webhook; verify HTTP 400 returned
NFR-ST-05	Pipeline Fault Tolerance	UC-03 View Analysis Results	AnalysisEngine	Inject analyzer failure; verify zero unhandled exceptions
NFR-ST-06	Graceful Parsing Failure	UC-02 Upload CV	PDFParser / AnalysisEngine	Upload scanned PDF; verify status='failed' and no crash
NFR-ST-07	Single-Page Results View	UC-03 View Analysis Results	React Frontend / Results Page	Verify score, ATS, suggestions, skills, and recommendations all appear on one page
NFR-ST-09	System Uptime	All Use Cases	All System Components	Verify system availability $\geq 99.5\%$ over monitoring period
NFR-ST-10	Container Deployment	All Use Cases	Docker / Docker Compose	Verify system starts correctly with docker-compose up
NFR-ST-13	Enumeration Safety	UC-08 Reset Password	AuthRouter	Submit existing and non-existing emails to /auth/forgot-password; verify identical HTTP 200 response body.
NFR-ST-14	Reset Code Security	UC-08 Reset Password	EmailService / AuthRouter / User Model	Verify generated code length and character set, 10-minute expiry, and 5-attempt limit.
NFR-ST-15	Password Reset Password Policy	UC-08 Reset Password	AuthRouter / hashing.py	Submit weak password and reused password; verify rejection. Submit valid new password; verify bcrypt hash stored.
NFR-SC-01	Analysis Pipeline Timing	UC-03 View Analysis Results	AnalysisEngine / BackgroundTasks	Measure pipeline completion time ≤ 10 seconds for standard 2-page PDF
NFR-SC-02	API Response Time	All Use Cases	FastAPI / Uvicorn	Measure read-only endpoint response time $\leq 500\text{ms}$ at p95 under normal load
NFR-SC-03	Database Query Performance	UC-03 View Analysis Results	SQLAlchemy / PostgreSQL	Measure single AnalysisResult retrieval time $\leq 200\text{ms}$
NFR-SC-05	Concurrent Upload Support	UC-02 Upload CV	CVRouter / BackgroundTasks	Simulate 50 concurrent uploads; verify no degradation

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Constraints

ID	Name	Use Case	Component / Module	Test Case
CON-TECH-01	Server Infrastructure	All Use Cases	Backend Container	Verify system runs on server with ≥ 2 GB RAM and 1 vCPU
CON-TECH-03	Supported File Formats	UC-02 Upload CV	CVService / PDFParser / TXTParser	Upload unsupported format (e.g., .docx); verify rejection
CON-TECH-06	Backend Framework	All Use Cases	FastAPI Application	Verify backend runs on FastAPI; check framework version in requirements.txt
CON-TECH-07	ORM and Migration Tool	All Use Cases	SQLAlchemy / Alembic	Run alembic upgrade head; verify migrations apply without error
CON-TECH-08	Frontend Framework	All Use Cases	React + Vite	Verify frontend builds successfully with npm run build
CON-TECH-09	Rate Limiting	UC-02 Upload CV / UC-03	slowapi / FastAPI Middleware	Exceed rate limit on upload endpoint; verify HTTP 429 returned
CON-ENV-01	Database Environment	All Use Cases	PostgreSQL	Verify production deployment uses PostgreSQL, not SQLite
CON-ENV-02	OpenAI API Dependency	UC-05 Request AI Summary	AnalysisService / OpenAI Client	Remove API key; verify system returns null for ai_summary gracefully
CON-ENV-05	SMTP Email Dependency	UC-08 Reset Password	EmailService / Settings	Configure SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL, SMTP_FROM_NAME, and SMTP_TLS; verify reset email delivery through Gmail SMTP.
CON-REG-01	Data Privacy	All Use Cases	All System Components	Verify no CV data transmitted to external services other than OpenAI and payment gateway
CON-REG-02	Licensing Compliance	All Use Cases	All Third-Party Libraries	Run license checker; verify all libraries are MIT, Apache 2.0, or equivalent

6. CONCLUSION

In this project, the CVision AI-powered CV Analyzer and Career Recommendation Platform has been analyzed and its requirements have been defined. During the requirements elicitation process, system needs were identified through stakeholder analysis, use case modeling, scenario development, and source code review. Functional requirements, non-functional requirements, constraints, and assumptions were documented in structured tables following the MoSCoW prioritization technique.

The system is designed to allow registered users to upload CV files, automatically analyze them through a 7-step sequential analysis pipeline (SectionDetector → SkillExtractor → KeywordScorer → ExperienceEvaluator → ATSChecker → ScoreCalculator → SuggestionGenerator), and receive actionable improvement suggestions, ATS compatibility scores, and career role recommendations. The platform also supports AI-generated CV summaries via the OpenAI GPT API for premium subscribers, subscription management via Stripe/lyzico payment gateway integration, an admin dashboard for user and log management, and a secure password reset workflow through Gmail SMTP email verification.

During the requirements analysis phase, the identified requirements were organized, refined, and validated. Various system models were created, including a use case diagram, entity-relationship data model, architecture class diagram, activity diagram, sequence diagram, state machine diagram, component and deployment diagrams, and a system architecture diagram. These models helped to clearly represent the system structure, behavior, and interactions between the frontend, backend, analysis engine, storage layer, and external services.

Some assumptions were made regarding the availability of external services such as the OpenAI API and Stripe payment gateway, the use of digital (text-layer) PDF files, and English-language CV content. The system is designed based on realistic constraints including the FastAPI and React technology stack, PyMuPDF for PDF parsing, and Docker-based containerized deployment. However, limitations such as lack of OCR support for scanned PDFs and English-only analysis dictionaries may affect usability for a broader user base.

In the future, the system can be improved by adding OCR support for scanned documents, extending skill and keyword dictionaries to support multiple languages, implementing real-time WebSocket-based analysis progress notifications, and scaling the deployment to support higher concurrent user loads through horizontal scaling and load balancing.

7. REFERENCES

- [1] A. M. Al-Aswad, H. M. Al-Qamati, and H. S. Abu. Alhaji, “*An Intelligent Resume Analysis and Job Recommendation System Using NLP*,” *The Journal of Comprehensive Sciences*, vol. 10, no. 38, pp. 2615-2625, 2026.
- [2] D. Olabanji, T. Fitch, and O. Matthew, “*Multi-tenancy in Cloud-native Architecture: A Systematic Mapping Study*,” *WSEAS Transactions on Computers*, vol. 22, 2023.
- [3] A. M. Aubaid and A. Mishra, “*A Rule-Based Approach to Embedding Techniques for Text Document Classification*,” *Applied Sciences*, vol. 10, no. 11, 2020.
- [4] A. Mahajan, A. Bhosale, B. Patel, and N. Gupta, “*Technical Skills Information Extraction From Resumes Using Advanced Natural Language Processing Model With Transfer Learning*,” *Journal of Information and Optimization Sciences*, 2025.
- [5] R. T. Fielding, “*Architectural Styles and the Design of Network-Based Software Architectures*,” Doctoral dissertation, University of California, Irvine, 2000.
- [6] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [7] L. Mathew, N. C. George, N. Linet, and N. K. Thomas, “*ATS BREAKER – A System for Comparing Candidate Resume and Company Requirements*,” *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 6, pp. 591-596, 2020.
- [8] G. Hendre, S. Bhiware, A. Darwajkar, and A. S. Mali, “*AI-Powered Resume Evaluation System*,” *International Journal of Advanced Research in Science, Communication and Technology*, vol. 5, no. 4, pp. 2290-2307, 2025.
- [9] Gaviya S and A. Joshy S, “*Smart Resume Scoring & Recommendations*,” *International Journal of Scientific Research and Engineering Development*, vol. 8, no. 2, pp. 1758-1767, 2025.
- [10] S. Oh et al., “*Architecture Design of Healthcare Software-as-a-Service Platform for Cloud-Based Clinical Decision Support Service*,” *Healthcare Informatics Research*, vol. 21, no. 2, pp. 102-1102, 2015.

8. DESCRIPTIONS, ABBREVIATIONS, AND DICTIONARY

8.1. Descriptions

Table 4.1. Descriptions.

Word or Concept	Description
Analysis Engine	The core processing component of CVision that orchestrates the sequential 7-step CV analysis pipeline.
AnalysisResult	A database record that stores the output of the analysis pipeline including overall score, ATS score, AI summary, suggestions, extracted skills, and career recommendations.
ATS (Applicant Tracking System)	Software used by employers to scan, sort, and rank CVs before human review. CVision measures CV compatibility with ATS systems.
BaseAnalyzer	Abstract base class that all analysis modules implement. Defines the analyze(context) interface for the pipeline.
Background Task	An asynchronous operation triggered by FastAPI's BackgroundTasks mechanism that runs the analysis pipeline without blocking the HTTP response.
CV (Curriculum Vitae)	A structured document presenting a candidate's professional experience, education, and skills for job applications.
JWT (JSON Web Token)	A signed, stateless token used for authenticating API requests. Encodes user identity and expiry claims.
Plan Type	User account tier in CVision: 'free' (limited monthly analyses), 'premium' (unlimited analyses + AI summary), 'admin' (system management).
RoleProfile	A pre-seeded database record describing a job role's expected skills and keywords, used for career recommendation matching.
SaaS (Software as a Service)	Cloud-hosted software delivery model where the application is accessed via a web browser without local installation.
Webhook	An HTTP callback mechanism used by Stripe/Iyzico to notify CVision of payment events in real time.
Yapay Zeka	Normalde insan zekasıyla gerçekleştirilebilecek işlemlerin öğrenebilen bilgisayar algoritmalarıyla gerçekleştirilmesidir.
EmailService	The backend service responsible for composing and delivering transactional emails (e.g., password reset codes) via Gmail SMTP using STARTTLS encryption on port 587.
Forgot Password	A password recovery process that allows an unauthenticated user to request a reset code using their registered email address.
SMTP	Simple Mail Transfer Protocol. CVision uses SMTP with STARTTLS to send password reset emails through Gmail.
STARTTLS	A protocol command that upgrades an existing SMTP connection to an encrypted TLS connection.
Password History	A stored record of recent password hashes used to prevent users from reusing their last 3 passwords during password reset.
Reset Code	A 5-character alphanumeric token generated by the server during a password reset request. It is stored in the database with a 10-minute expiry and a maximum of 5 verification attempts before it is invalidated.

8.2. English and Turkish Abbreviations

Table 4.2. Turkish abbreviations.

Abbreviation	Meaning
CD/CI	Yapay Zeka
VT	Veritabanı

8.3. Turkish Dictionary

Table 4.3. Turkish dictionary.

Turkish	English Equivalent
Gereksinim / Gerekler	Requirements
Yazılım Mimarisi	Software Architecture
Analiz Motoru	Analysis Engine
Özgeçmiş	Curriculum Vitae (CV)
Yapay Zeka	Artificial Intelligence (AI)
Kullanım Senaryosu	Use Case
İşlevsel Gereksinim	Functional Requirement
İşlevsel Olmayan Gereksinim	Non-Functional Requirement
Kısıt	Constraint
Paydaş	Stakeholder
Doğrulama	Validation / Verification
İzlenebilirlik	Traceability
Veri Modeli	Data Model